The Security of ML-DSA against Fault-Injection Attacks *

Haruhisa Kosuge[1] and Keita Xagawa[2][0000−0002−6832−9940]

[1] NTT Social Informatics Laboratories, Japan
hrhs.kosuge@ntt.com
[2] Technology Innovation Institute, UAE
keita.xagawa@tii.ae

Abstract. Deterministic signatures are often used to mitigate the risks associated with poor-quality randomness, where the randomness in the signing process is generated by a pseudorandom function that takes a message as input. However, some studies have shown that such signatures are vulnerable to fault-injection attacks. To strike a balance, recent signature schemes often adopt "hedged" randomness generation, where the pseudorandom function takes both a message and a nonce as input. Aranha et al. (EUROCRYPT 2020) investigated the security of hedged Fiat-Shamir signatures against 1-bit faults and provided a security proof for specific classes of bit-tampering attacks. Grilo et al. (ASIACRYPT 2021) extended this proof to the quantum random oracle model. Last year, NIST standardized the lattice-based signature scheme ML-DSA, which adopts the hedged Fiat-Shamir *with aborts*. However, existing security proofs against bit-tampering faults do not directly apply, as Aranha et al. left this as an open problem.
To address this gap, we analyze the security of ML-DSA against multi-bit fault-injection attacks. We provide a formal proof of security for a specific class of faults at the inputs and outputs of internal functions, showing that faults at these points cannot be exploited. Furthermore, to highlight the infeasibility of stronger fault resilience, we survey key-recovery attacks that exploit signatures generated under fault injection at the other intermediate points.

1 Introduction

Post-quantum Digital Signature and ML-DSA: Digital signatures are essential for ensuring the integrity and authenticity of digital communications. The standard security notion for digital signatures is existential unforgeability against a chosen-message attack (EUF-CMA) [GMR88]. Roughly speaking, a signature scheme is considered EUF-CMA-secure if no efficient adversary can forge a signature, even with access to a signing oracle.

The emergence of quantum computers has raised significant concerns about the security of traditional digital signatures due to Shor's algorithm [Sho94]. Consequently, post-quantum cryptography (PQC) has garnered increasing attention. In 2022, NIST decided to standardize three post-quantum digital signatures: Falcon, Dilithium, and SPHINCS+. Dilithium and SPHINCS+ have already been standardized as ML-DSA (FIPS 204) [Nat24a] and SLH-DSA (FIPS 205) [Nat24b].

This work focuses on ML-DSA, which follows *Fiat-Shamir with aborts* [Lyu09, Lyu12]. The Fiat-Shamir transform [FS87] is widely used to convert interactive identification (ID) schemes to non-interactive digital signature schemes. A typical ID scheme consists of the following three steps:

Commitment: The prover generates a commitment w and a state st, then sends w to the verifier.
Challenge: The verifier selects a random challenge c and sends it to the prover.
Response: Upon receiving c, the prover computes a response z from (c, st).

The verifier then checks that a transcript (w, c, z) is valid. The Fiat-Shamir transform gives a non-interactive signature scheme by replacing the verifier's random challenge c with a challenge generated by a hash function H. Given a message m, it computes the challenge $c := \mathsf{H}(w, m)$. In the Fiat-Shamir with aborts paradigm, the response function may output $\perp$ (indicating abort) and repeat the procedure until a valid transcript is produced.

Deterministic Signature and Fault-injection Attack: While the quality of randomness in cryptographic algorithms is critical, it is non-trivial to ensure good randomness in the real environment. To mitigate this situation, we sometimes adopt the deterministic signature, whose signing algorithm produces randomness by using a pseudorandom function (PRF) with a secret key and a message to be signed as input. See, e.g., EdDSA [BDL+11] and deterministic variants of DSA/ECDSA [Por13]. If the secret key is correctly chosen, then the deterministic signature scheme is secure even without randomness.

Unfortunately, a fault-injection attack is a significant threat to deterministic signature schemes, particularly those based on the Fiat-Shamir paradigm. A well-known vulnerability of deterministic Fiat-Shamir signatures is the *differential fault attack* or *special soundness attack* (e.g., [RP17, PSS+18, ABF+18, SB18] against EdDSA and [BP18, AWK+25] against deterministic Dilithium/ML-DSA), in which faults are injected during multiple signing attempts for the same message. Such fault injections can generate multiple challenge-response pairs associated with a single commitment. The special soundness of the ID scheme leads to the disclosure of the private key. As such, fault-injection attacks pose a critical threat to deterministic Fiat-Shamir signatures, necessitating mitigation.

Hedging: Bellare and Tackmann [BT16] introduced a technique called *hedging* for digital signatures, designed to mitigate failures in randomness generation. This technique has been adopted in several Fiat-Shamir signatures, such as XEdDSA [Per16]. Aranha, Orlandi, Takahashi, and Zaverucha [AOTZ20] formalized this concept as *hedged Fiat-Shamir*. Hedging modifies the signing algorithm by introducing a small amount of randomness, enhancing its fault resilience. Specifically, it introduces a nonce for the PRF to prevent the same commitments from being generated repeatedly, thereby mitigating the special soundness attack. ML-DSA offers the *hedged* variant for signature generation in addition to the *deterministic* variant, which sets the nonce 0. FIPS 204 [Nat24a] strongly recommends the hedged variant in environments susceptible to fault-injection attacks, as it does not impact the verification process and maintains interoperability between the variants.

Previous Works: The Fiat-Shamir with aborts is known EUF-CMA-secure in the (quantum) random oracle model ((Q)ROM) [AFLT16, KLS18, BBD+23, DFPS23]. On the security against fault-injection attacks, two studies examined the Fiat-Shamir signatures.[3] Aranha et al. [AOTZ20] introduced the EUF-fCMNA (existential unforgeability under faults, chosen message and nonce attacks) model, a fault-aware extension of EUF-CMA designed to analyze Fiat-Shamir signatures. This model allows an adversary to inject a single-bit fault into the inputs or outputs of any intermediate functions. They showed that the hedged Fiat-Shamir is EUF-fCMNA-secure in the ROM for some types of faults. Grilo et al. [GHHM21] extended the result to the QROM, demonstrating equivalent security guarantees in the quantum setting.

Despite the above advancements, the security of *the hedged Fiat-Shamir with aborts* remains unanalyzed, even though Aranha et al. [AOTZ20, ePrint] highlighted this as one of the open problems. Moreover, due to structural differences from the hedged Fiat-Shamir, existing analyses cannot be directly applied to ML-DSA. Accordingly, we propose the following research question:

To what extent is ML-DSA secure against fault-injection attacks?

1.1 Contribution

Regarding our research question, we prove that hedging in ML-DSA mitigates certain classes of fault-injection attacks, and we also show that faults on other function inputs/outputs remain unmitigated, as they lead to concrete key-recovery attacks. Note that faults occurring inside intermediate functions are outside the scope of our contribution, even if such attacks are feasible in the real world. We divide the contribution into three points.

Security Model for ML-DSA against Fault-injection Attacks: We modify the EUF-fCMNA security notion defined by Aranha et al. [AOTZ20] to the specific structure of ML-DSA and also generalize the model to capture a broader range of fault-injection attacks. Given the looped structure of Fiat-Shamir with aborts, we allow the adversary to choose the iteration in which a fault is injected. Moreover, we

[3] Fischlin and Günther [FG20] gave a general treatment of hedged signatures. In contrast, this work is based on research that targets Fiat-Shamir signatures.

Table 1: Summary of our results. "✓" indicates that fault-resilience is proven, and "×" indicates a key-recovery attack.

Function	Symbol	Interface	Secure
Transformation of private key	T_{sk}	Input Output	× ×
Message representation function	H_{μ}	Input Output	✓ ✓
Hedged extractor	H_{he}	Input (key) Input (others) Output	✓ × ✓
Expand mask	$\mathsf{H}_{\mathsf{em}}^{(\ell)}$	Input (counter) Input (seed) Output	× ✓ ×
Commitment	P_1	Input Output	× ×
Challenge hash	H_{ch}	Input Output	✓ ✓
Challenge sampling function (SampleInBall)	Sb	Input Output	✓ ×
Response	P_2	Input Output	× ×
Abort procedure	Ab	Input Output	× ✓

incorporate the per-loop randomness generation unique to ML-DSA and model fault injections targeting the inputs and outputs of each associated function. Finally, we generalize the original fault model, which considered only single-bit faults, to allow for up to (non-contiguous) t-bit faults. In general, if the target bits are specified, multi-bit faults are more powerful than single-bit faults. While this can be more difficult to achieve, it is feasible in practice (e.g., [SHS16, CGV+21]). We note that if the target bits are not specified in detail, multi-bit faults are easier to introduce (e.g., instruction skip, register randomization, and register zeroization). Thus, security against multi-bit faults is more desirable.

Security Proof of ML-DSA under Our Model: We prove that ML-DSA remains EUF-fCMNA-secure in the QROM when an adversary injects faults into any of the inputs or outputs of the functions marked with checkmarks in Table 1, under the assumption that the number of tampered bits is small. Fig. 1 provides a visual representation of the results of Table 1.[4] Moreover, it captures the fault-free case as a special instance by setting the number of tampered bits to 0. To the best of our knowledge, this is the first security proof that fully conforms to the ML-DSA specification. We note that, to show the security, we assume that the underlying simplified ML-DSA is EUF-NMA-secure in the QROM, where EUF-NMA stands for Non-Message Attack. In the simplified ML-DSA, per-loop randomness $\boldsymbol{y}$ in Fig. 1 is freshly sampled.[5] The EUF-NMA security of the simplified ML-DSA is justified by the hardness of lattice problems (the MLWE and the Self Target MSIS problem) [KLS18, JMW24] or the security of the underlying ID scheme [DFMS19].

The main technical challenge in the security proof lies in the structured generation of randomness during signature generation. In the Fiat-Shamir with aborts framework, the signing algorithm requires fresh randomness for each loop iteration. When such randomness is derived from a single function, existing proof techniques [AOTZ20, GHHM21] can be directly applied. However, in ML-DSA, randomness is

[4] In our model, the abort procedure Ab outputs a response if it does not abort, and $\perp$ otherwise, with $\perp$ assumed unchanged under faults. If instead the output is modeled as a success bit that a fault can flip, faults on the output of Ab lead to key-recovery attacks.

[5] In addition, the public key is uncompressed.

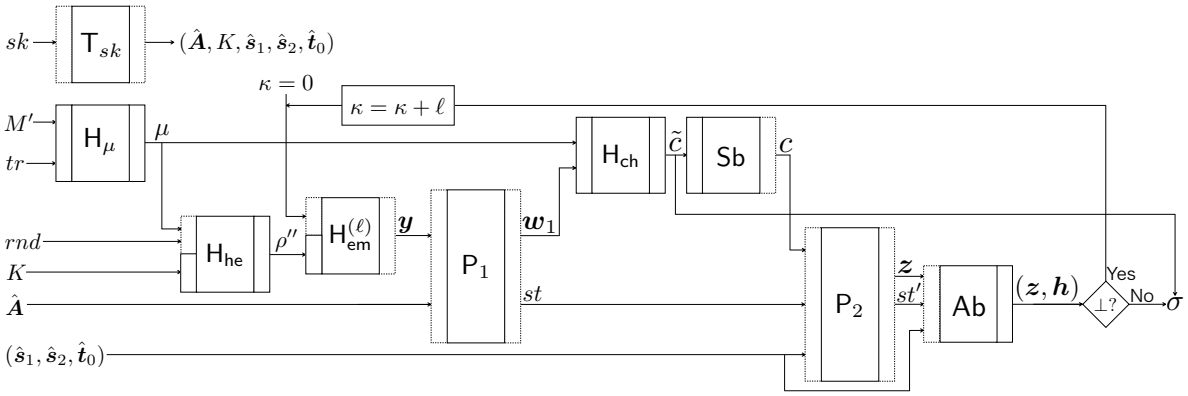

Fig. 1: A diagram illustrating the signature generation of ML-DSA. A function's left (resp., right) rectangle is solid if its input (resp., output) is proven fault-resilient. The dashed rectangles indicate the absence of proven fault-resilience.

generated through multiple dedicated functions (see $\boldsymbol{y}$ in Fig. 1). This structural deviation necessitates novel techniques. In particular, careful analysis of the collision resistance of random functions under fault injection is required. Moreover, the generalization from single-bit to multi-bit fault models further complicates the analysis. See Section 1.2 below for details.

Infeasibility to Prove Stronger Fault Resilience: Under our model, where faults are restricted to function inputs and outputs, we cannot prove stronger fault resilience for ML-DSA beyond our proof. To demonstrate it, we theoretically analyze the possibility of mounting an attack by injecting faults into intermediate values for which resilience cannot be proven. The faults marked with × in the "Secure" column of Table 1 can lead to a successful key recovery. Our analysis based on the existing SCA/FIA-based key-recovery attacks classifies all types of faults as ✓ or ×; thus, we conclude that proving stronger fault resilience for ML-DSA is infeasible.

We have found that there is a notable vulnerability inherent to ML-DSA (and the Fiat-Shamir with aborts in general), which is the class of attacks that aim to bypass the abort conditions and induce the output of responses that leak information about the secrets.

Implications of Our Results: An important finding of our study is that the inputs and outputs of internal functions in ML-DSA's signature generation can be categorized into two types: those unaffected by faults and those that lead to key-recovery attacks. This distinction helps identify which function interfaces merit stronger tamper-proofing, and provides a useful starting point for refining practical countermeasures.

In addition, our security proof yields two key observations. First, the dominant term in the security bound arises from the relatively short length of the key K used for hedging, which becomes non-negligible when multiple bits are tampered. As practical countermeasures, one may employ longer K and/or implement protections against multi-bit tampering. Second, among the faults not covered by our proof, those targeting the output of the challenge sampling function can be made provable by introducing a validity check to ensure that the challenge lies within the challenge space. Since this is a minor modification, we encourage its adoption in practice. We note that if a fault is injected after the validity check within the internal computation of the response function, such an attack cannot be prevented.

1.2 Technical Overview

Following the strategy of Aranha et al. [AOTZ20], we define an intermediate security notion, EUF-fCMA security (existential unforgeability under faults, chosen message attacks), for the simplified ML-DSA. We then establish two reductions: EUF-NMA ⇒ EUF-fCMA and EUF-fCMA ⇒ EUF-fCMNA, where $X \Rightarrow Y$ denotes a reduction of Y to X. This approach is taken because the latter reduction explicitly highlights the security improvement brought by the hedging.

EUF-NMA $\Rightarrow$ EUF-fCMA*:* With some modifications, the EUF-NMA adversary can simulate the EUF-fCMA game. Its overall strategy is to simulate tampered signatures while ensuring it can win its game using the message/signature pair (M^*, σ^*) finally submitted by the EUF-fCMA adversary.

We first introduce *special no-abort honest-verifier zero-knowledge (snaHVZK)*. Informally, we say that ID is snaHVZK if there exists an efficient simulator that, given a public key pk and a challenge c, can perfectly simulate honestly generated no-aborting transcripts.[6] We prove that the underlying ID scheme of simplified ML-DSA is snaHVZK by extending the proof of no-abort HVZK for Dilithium [KLS18]. Note that the snaHVZK property is essential for simulating the signing oracle with faults because the challenge will be tampered by the adversary, as Aranha et al. [AOTZ20] introduced special HVZK.

To enable the simulation using the snaHVZK simulator, we employ two techniques in the QROM to reprogram the random function. Specifically, $\tilde{c}$ chosen uniformly at random must satisfy $\mathsf{H}_{\mathsf{ch}}(\boldsymbol{w}_1, \mu) = \tilde{c}$, where $\tilde{c}$ is a hash value used for challenge sampling and $(\boldsymbol{w}_1, \boldsymbol{z}, \boldsymbol{h})$ is an output of the snaHVZK simulator. The technique of [GHHM21] enables such adaptive reprogramming, assuming that $\boldsymbol{w}_1$ has sufficiently high min-entropy. While t-bit fault injections reduce the min-entropy, the degradation is bounded by t bits; therefore, for small values of t, adaptive reprogramming is feasible. However, the adaptive reprogramming using [GHHM21] requires reprogramming even for *aborted* transcripts. Since the snaHVZK simulator only generates no-aborting transcripts, the reprogramming corresponding to aborted transcripts is infeasible. To address this challenge, we invoke the O2H lemma [AHU19] to avoid such reprogramming following the approaches of [BBD+23, KX24a]. Importantly, we extend the original O2H lemma of [AHU19] to accommodate the case where reprogramming is performed adaptively.[7] Then, the EUF-NMA adversary can simulate tampered signatures using the snaHVZK simulator.

We can ensure that the EUF-NMA adversary can win its game using the output of the EUF-fCMA adversary by considering the specification that the challenge hash H_{ch} takes the message representative μ instead of the encoded message M' (i.e., the message concatenated with a context string). Suppose the EUF-fCMA adversary outputs (M^*, σ^*) and $\mu^* = \mathsf{H}_\mu(tr, M^*)$ has not been used in any signing query. In that case, when the EUF-fCMA adversary wins the game, (M^*, σ^*) allows the EUF-NMA adversary to win as well, since (M^*, σ^*) is verified using the non-reprogrammed $\hat{\mathsf{H}}_{\mathsf{ch}}$, which the EUF-NMA adversary can access. Note that the adversary can inject faults of up to t bits into the output of H_μ. Letting ϕ be a fault function, the probability that $\mu^* = \mathsf{H}_\mu(tr, M^*)$ has been used in the signing queries is bounded by the probability of finding (ϕ, M') such that $\mathsf{H}_\mu(tr, M^*) = \phi(\mathsf{H}_\mu(tr, M'))$ and $M' \neq M^*$. This probability is derived by extending Zhandry's bound on collision probability in the QROM [Zha15].

EUF-fCMA $\Rightarrow$ EUF-fCMNA*:* The EUF-fCMA adversary against the simplified ML-DSA can simulate the EUF-fCMNA game, once the signing process in the EUF-fCMNA game is made fully probabilistic. We sequentially randomize the outputs of the hedged extractor H_{he} and the expand mask $\mathsf{H}^{(\ell)}_{\mathsf{em}}$, which are used to compute $\boldsymbol{y}$ as shown in Fig. 1, as follows.

We can randomize the output of the hedged extractor H_{he}, denoted by ρ'', using semi-classical O2H lemma [AHU19]. We puncture inputs used to generate ρ'', thereby preventing the adversary from querying these inputs. This ensures that the adversary cannot distinguish whether the output of H_{he} is randomized. Here, we assume that the inputs to H_{he} do not collide, allowing its outputs to be replaced with fresh random values. The probability of finding a collision on the input is bounded by the probability of finding $(\phi_1, \phi_2, (tr_1, M'_1), (tr_2, M'_2))$ such that $\phi_1(\mathsf{H}_\mu(tr_1, M'_1)) = \phi_2(\mathsf{H}_\mu(tr_2, M'_2))$, which is upper-bounded similarly as the case where ϕ is applied only to one side.

Applying the above technique again, the output $\boldsymbol{y}$ of the expand mask $\mathsf{H}^{(\ell)}_{\mathsf{em}}$ is randomized, where $\mathsf{H}^{(\ell)}_{\mathsf{em}}$ computes H_{em} in parallel ℓ times using a seed ρ'' and counters $\kappa, \ldots, \kappa+\ell-1$ as input. We need to assume that all inputs to H_{em} must be distinct. We consider the probability of collision where $\phi_1(\rho''_1) = \phi_2(\rho''_2)$ for random (ρ''_1, ρ''_2) and fault functions (ϕ_1, ϕ_2). Whenever a pair (ρ''_1, ρ''_2) differs in at most $2t$ bits, there exists (ϕ_1, ϕ_2) such that $\phi_1(\rho''_1) = \phi_2(\rho''_2)$. Thus, the probability that the inputs to H_{em} are not distinct can be bounded by the probability that two random values collide after allowing up to $2t$-bit differences.

[6] The simulator outputs (w, z) for a no-aborting transcript or $\perp$ for an aborted one.

[7] Although the same result follows from the gentle measurement lemma [BBD+23], we use the O2H lemma for its common use in prior works [DFPS23, Xag24].

1.3 Open Problem

Since the current hedging technique cannot mitigate some fault-injection attacks, further research is needed to develop new approaches to enhance the fault resilience of ML-DSA specifically or Fiat-Shamir with aborts in general. For instance, to prevent an invalid response from being output without aborting, one may consider modifying the algorithm to include a verification step before outputting the signature as implied in [BDL01, AOCVCG25]. However, any such technique requires further analysis under our security model to determine which types of fault injections can actually be prevented.

1.4 Organization

Section 2 introduces the notations, terminologies, definitions, and proof techniques. We review the specification of ML-DSA in Section 3. For the security notion defined in Section 4, we establish the security proof for ML-DSA in Section 5. In Section 6, we demonstrate that proving stronger fault resilience is infeasible. In the appendices, we survey fault-injection attacks on ML-DSA, provide missing definitions and proofs, explain the intuition behind the fault resilience of ML-DSA, and present a concrete parameter analysis.

2 Preliminary

2.1 Notations and Terminology

We use calligraphic font for sets and adversaries, bold font for vectors and matrices, sans-serif font for functions and games, and small capitals for oracles.

For $m, n \in \mathbb{N}$, we let $[n] = \{1, 2, \ldots, n\}$ and $[m, n] = \{m, m+1, \ldots, n\}$. For a finite set $\mathcal{X}$, $|\mathcal{X}|$ is the cardinality of $\mathcal{X}$. We denote uniform sampling by $x \leftarrow_{\$} \mathcal{X}$ and sampling from a distribution D over $\mathcal{X}$ by $x \leftarrow D$. We denote the set of all functions with a domain $\mathcal{X}$ and a range $\mathcal{Y}$ by $\mathsf{Func}(\mathcal{X}, \mathcal{Y})$.

If a function F is deterministic (resp., probabilistic), we write $y := \mathsf{F}(x)$ (resp., $y \leftarrow \mathsf{F}(x)$). We denote by $y \leftarrow \mathcal{A}^{\mathrm{O}}(x)$ (resp., $y \leftarrow \mathcal{A}^{|\mathrm{O}\rangle}(x)$) probabilistic computations of $\mathcal{A}$ on input x with a classical (resp., quantum) oracle access to an oracle O. When representing a probabilistic function $\mathsf{F}(x)$ as a deterministic function with explicit randomness r, we write it as $\mathsf{F}(x; r)$. For a random function H, we denote by $\mathsf{H}^{x^* \mapsto y^*}$ a function such that $\mathsf{H}^{x^* \mapsto y^*}(x) = \mathsf{H}(x)$ for $x \neq x^*$ and $\mathsf{H}^{x^* \mapsto y^*}(x^*) = y^*$. The notation $G^{\mathcal{A}} = y$ denotes an event in which a game G played by $\mathcal{A}$ returns y. For i-th game G_i, we denote W_i as an event $G_i^{\mathcal{A}} = \top$.

We follow the notation of FIPS 204 [Nat24a, Section 2.3]. Let $R = \mathbb{Z}[x]/(x^{256} + 1)$ and $R_q = \mathbb{Z}_q[x]/(x^{256} + 1)$. For a positive integer η, S_η denotes the set of polynomials in R whose coefficients have absolute values at most η; that is $S_\eta = \{a = \sum_i a_i x^i \in R : \forall i, a_i \in [-\eta, \eta]\}$. We also define S'_η as $\{a = \sum_i a_i x^i \in R : \forall i, a_i \in [-\eta + 1, \eta]\}$. Note that $S_{\eta-1} \subseteq S'_\eta$. Let $\mathbb{B} = \{0, 1, \ldots, 255\}$ be the set of integers that represents a byte. In the context of fault-injection attacks, we use an overline to denote a tampered value, distinguishing it from its original counterpart.

2.2 Digital Signature

We define the syntax of digital signature schemes as follows.

Definition 1 (Digital Signature). *A digital signature scheme* Sig *consists of three algorithms:*

- $\mathsf{KeyGen}(1^\lambda)$*: This algorithm takes* 1^λ*, where* λ *is the security parameter, as input and outputs a public key* pk *and a private key* sk*.*
- $\mathsf{Sign}(sk, m)$*: This algorithm takes a private key* sk *and a message* m *as input and outputs a signature* σ*.*
- $\mathsf{Verify}(pk, m, \sigma)$*: This algorithm takes a public key* pk*, a message* m*, and a signature* σ *as input, and deterministically outputs its decision* $\top$ *or* $\bot$*.*

We require statistical correctness; that is, for any message $m \in \{0,1\}^*$*, we have*

$$\Pr\left[(pk, sk) \leftarrow \mathsf{KeyGen}(1^\lambda), \sigma \leftarrow \mathsf{Sign}(sk, m) : \mathsf{Verify}(pk, m, \sigma) = \top\right] = 1 - \mathsf{negl}(\lambda),$$

for some negligible function negl*.*

$\mathsf{Trans}(sk, c)$
1 $(w, st) \leftarrow \mathsf{P}_1(sk)$
2 $z \leftarrow \mathsf{P}_2(sk, c, st)$
3 **if** $z = \perp$ **then return** $\perp$
4 **else return** (w, z)

Fig. 2: Honest generation of transcript

2.3 Identification (ID) Scheme

We first give the syntax of ID schemes.

Definition 2 (Identification (ID) Scheme). *An ID scheme, denoted as* ID*, consists of four algorithms and a space* $\mathcal{C}$*:*

$\mathsf{Gen}(1^\lambda)$*: This algorithm takes* 1^λ*, where* λ *is the security parameter, as input and outputs a public key* pk *and a private key* sk*.*
$\mathsf{P}_1(sk)$*: This algorithm takes a private key* sk *as input and outputs a commitment* w *and a state* st*.*
$\mathcal{C}$*: This space is where the challenge* c *is sampled.*
$\mathsf{P}_2(sk, c, st)$*: This algorithm takes a private key* sk*, a challenge* $c \in \mathcal{C}$*, and a state* st *as input and deterministically outputs a response* z *or* $\perp$*.*
$\mathsf{V}(pk, w, c, z)$*: This algorithm takes a public key* pk*, a commitment* w*, a challenge* $c \in \mathcal{C}$*, and a response* z *as input and deterministically outputs its decision* $\top$ *or* $\perp$*.*

We refer to a triple (w, c, z) *as a transcript.*

We say that ID has correctness error ε [KLS18, Definition 2.3] if for all $(pk, sk) \leftarrow \mathsf{Gen}(1^\lambda)$, the following holds:

– $\Pr[(w, st) \leftarrow \mathsf{P}_1(sk), c \leftarrow_\$ \mathcal{C}, z \coloneqq \mathsf{P}_2(sk, c, st) : z = \perp] \leq \varepsilon$ and
– $\Pr[(w, st) \leftarrow \mathsf{P}_1(sk), c \leftarrow_\$ \mathcal{C}, z \coloneqq \mathsf{P}_2(sk, c, st) : \mathsf{V}(pk, w, c, z) = \top | z \neq \perp] = 1$.

Also, we say that ID has (α, δ)-min-entropy [DFPS23, Definition 5] if

$$\Pr\big[(pk, sk) \leftarrow \mathsf{Gen}(1^\lambda) : H_\infty(w | (w, st) \leftarrow \mathsf{P}_1(sk)) \geq \alpha\big] \geq 1 - \delta.$$

In this paper, we only consider *commitment-recoverable* ID scheme, which is equipped with a commitment-recovery algorithm Rec.

Definition 3 (Commitment-Recoverable ID [KLS18, Def.2.4]). *We call an ID scheme* commitment-recoverable *if for any* (pk, sk) *generated by* Gen*, any challenge* c*, and any response* z*, there exists a unique* w *such that* $\mathsf{V}(pk, w, c, z) = 1$*. We also need a DPT algorithm* Rec *that computes* w*, that is,* $w \coloneqq \mathsf{Rec}(pk, c, z)$*.*

Next, we introduce an honest-verifier zero knowledge (HVZK) property of ID. Since we consider the security against fault-injection attacks, the challenge will be manipulated by the adversary. Thus, we employ *special* HVZK [AOTZ20], where the simulator takes a challenge as input.[8] Moreover, since we consider ML-DSA, which adopts Fiat-Shamir with abort, we also adopt no-abort HVZK (naHVZK), as formalized in [KLS18, AFLT16]. The following definition merges these two definitions.

Definition 4 (Special No-Abort HVZK). *We call an ID scheme* special no-abort HVZK (snaHVZK) *if there exists a PPT simulator* Sim *that takes* pk *and* c *as input and outputs* (w, z) *or* $\perp$ *such that, for any* (pk, sk) *generated by* Gen *and any* $c \in \mathcal{C}$*, the simulator perfectly simulates the distribution of* Trans *on input* sk *and* c*,*[9] *where* Trans *is defined as in Fig. 2.*

[8] In the original HVZK, the simulator outputs a challenge.
[9] We only consider the perfect case, as it can be proven in ML-DSA, while [KLS18] considered the statistical case and [AOTZ20] considered the statistical/computational cases.

GAME AR_b

$\mathsf{H}_0 \leftarrow_{\$} \mathsf{Func}(\mathcal{X}, \mathcal{Y})$
$\mathsf{H}_1 := \mathsf{H}_0$
$b^* \leftarrow \mathcal{A}^{|\mathsf{H}_b\rangle, \textsc{Repro}}()$
return b^*

$\textsc{Repro}(D)$

$(x, x') \leftarrow D$
$y \leftarrow_{\$} \mathcal{Y}$
$\mathsf{H}_1 := \mathsf{H}_1^{x \mapsto y}$
return (x, x')

Fig. 3: AR (Adaptive Reprogramming) game

2.4 Quantum Random Oracle Model (QROM) and Proof Techniques

In the ROM/QROM, a hash function $\mathsf{H}\colon \mathcal{X} \to \mathcal{Y}$ is modeled as a random function $\mathsf{H} \leftarrow_{\$} \mathsf{Func}(\mathcal{X}, \mathcal{Y})$. The adversary makes queries to the random oracle (random oracle queries) to compute the hash values. In the ROM, the challenger can choose $y \leftarrow_{\$} \mathcal{Y}$ and program $\mathsf{H} := \mathsf{H}^{x \mapsto y}$ for queried x on-the-fly instead of choosing $\mathsf{H} \leftarrow_{\$} \mathsf{Func}(\mathcal{X}, \mathcal{Y})$ at the beginning (lazy sampling technique). In the QROM, the adversary makes queries to H in a superposition of many different values, e.g., $\sum_x \alpha_x |x\rangle |y\rangle$. The challenger computes H and gives a superposition of the results to the adversary, $\sum_x \alpha_x |x\rangle |y \oplus \mathsf{H}(x)\rangle$. Due to the nature of superposition queries and other constraints of quantum computation, traditional techniques in the ROM cannot be directly applied to the QROM. However, recent advancements in QROM research have led to the discovery of many proof techniques. This section introduces the techniques used in this paper.

Tight Adaptive Reprogramming [GHHM21]: Let $\mathsf{H}_0, \mathsf{H}_1\colon \mathcal{X} \to \mathcal{Y}$ be random functions. Fig. 3 shows a game called AR (Adaptive Reprogramming) game, in which the adversary $\mathcal{A}$ attempts to distinguish H_0 (no reprogramming) from H_1 (reprogrammed by $\textsc{Repro}$). For a reprogramming query D, that is, a distribution of reprogramming point x and its side information x', it chooses $(x, x') \leftarrow D$ and $y \leftarrow_{\$} \mathcal{Y}$, reprograms H_1 as $\mathsf{H}_1^{x \mapsto y}$, and gives (x, x') to $\mathcal{A}$. A distinguishing advantage of the AR game is defined by $\mathrm{Adv}^{\mathrm{AR}}_{\mathsf{H}}(\mathcal{A}) = \left|\Pr\left[\mathsf{AR}_0^{\mathcal{A}} = 1\right] - \Pr\left[\mathsf{AR}_1^{\mathcal{A}} = 1\right]\right|$.

Lemma 1 (Tight Adaptive Reprogramming [GHHM21, Theorem 1], simplified). *For any quantum* AR *adversary $\mathcal{A}$ issuing at most q_{R} classical reprogramming queries and q_{H} (quantum) random oracle queries to H_b, the distinguishing advantage of the* AR *game is bounded by*

$$\mathrm{Adv}^{\mathrm{AR}}_{\mathsf{H}}(\mathcal{A}) \le \frac{3}{2} \sum_{i \in [q_{\mathsf{R}}]} \sqrt{q_{\mathsf{H}} \cdot p^{(i)}_{\mathsf{max}}},$$

where we define $p^{(i)}_{\mathsf{max}} := \mathbb{E} \max_{\hat{x}} \Pr_{(x,x') \leftarrow D_i}[x = \hat{x}]$ with the expectation taken over $\mathcal{A}$'s behavior up to the i-th query.

Semi-classical O2H Technique [AHU19]: We define *punctured oracle* following [BHH+19].

Definition 5 (Punctured Oracle [BHH+19, Definition 1]). *Let $\mathcal{S} \subset \mathcal{X}$ be a set and $\mathsf{f}_{\mathcal{S}}\colon \mathcal{X} \to \{0,1\}$ be a predicate that returns 1 if and only if $x \in \mathcal{S}$.* Punctured oracle $\mathsf{H}\backslash\mathcal{S}$ *(H punctured by $\mathcal{S}$) of $\mathsf{H}\colon \mathcal{X} \to \mathcal{Y}$ runs as follows: on input x, computes whether $x \in \mathcal{S}$ in an auxilliary qubit $|\mathsf{f}_{\mathcal{S}}(x)\rangle$, measures $|\mathsf{f}_{\mathcal{S}}(x)\rangle$, runs $\mathsf{H}(x)$, and returns the result. Let* **find** *be an event that any of measurements of $|\mathsf{f}_{\mathcal{S}}(x)\rangle$ returns* 1.

Given $\sum_x \alpha_x |x\rangle |y\rangle$, the punctured oracle $\mathsf{H}\backslash\mathcal{S}$ computes $\sum_x \alpha_x |x\rangle |y\rangle |\mathsf{f}_{\mathcal{S}}(x)\rangle$ and measures the third register. If the result is 0, then the query is transformed to $\sum_{x \notin \mathcal{S}} \alpha_x |x\rangle |y\rangle |0\rangle$ and the punctured oracle $\mathsf{H}\backslash\mathcal{S}$ returns $\sum_{x \notin \mathcal{S}} \alpha_x |x\rangle |y \oplus \mathsf{H}(x)\rangle$ to the adversary. If the results is 1 (and thus, **find** $= \top$ holds), $\mathsf{H}\backslash\mathcal{S}$ returns $\sum_{x \in \mathcal{S}} \alpha_x |x\rangle |y \oplus \mathsf{H}(x)\rangle$ to the adversary.

The following lemma provides a bound on the change in advantage caused by puncturing the oracle, based on the probability that **find** $= \top$.

Lemma 2 (Semi-classical O2H Technique [AHU19, Theorem 1], simplified). *Let $\mathcal{S} \subset \mathcal{X}$ be random. Let $\mathsf{H}_0, \mathsf{H}_1\colon \mathcal{X} \to \mathcal{Y}$ be random functions satisfying $\forall x \in \mathcal{X}$, $\mathsf{H}_0(x) = \mathsf{H}_1(x)$. Let z be a random bitstring. ($\mathcal{S}$, H_0, H_1, and z are taken from arbitrary joint distribution.) For any quantum adversary $\mathcal{A}$ issuing at most q (quantum) random oracle queries to H_0, we have*

$$\left|\Pr\left[1 \leftarrow \mathcal{A}^{|\mathsf{H}_0\rangle}(z)\right] - \Pr\left[1 \leftarrow \mathcal{A}^{|\mathsf{H}_0\backslash\mathcal{S}\rangle}(z)\right]\right| \le \sqrt{(q+1)\Pr\left[\mathcal{A}^{|\mathsf{H}_1\backslash\mathcal{S}\rangle}(z) : \textbf{find} = \top\right]}.$$

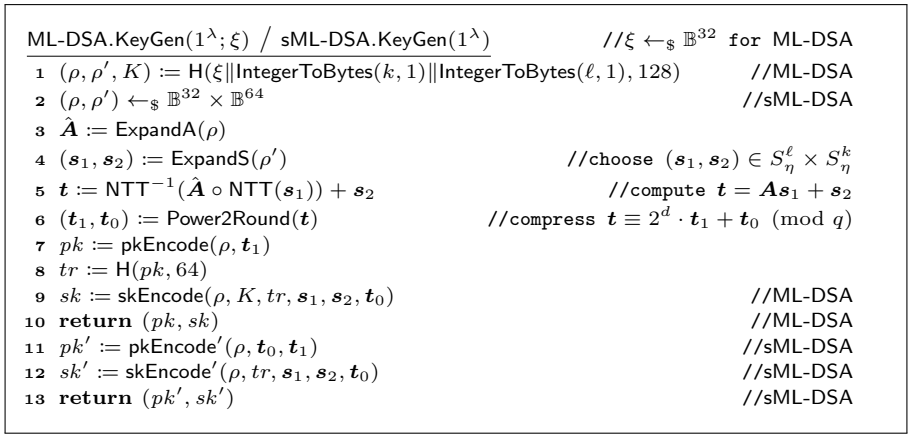

Fig. 4: Key generation of ML-DSA and simplified ML-DSA

Combining Lemma 2 with the following, we have a concrete bound.

Lemma 3 (Search in Semi-classical Oracle [AHU19, Theorem 2 and Corollary 1]). *Let $\mathcal{A}$ be a quantum adversary issuing at most q (quantum) random oracle queries to H_1. Let $\mathcal{B}^{|\mathsf{H}_1\rangle}(z)$ be an algorithm that runs as follows: Picks $i \leftarrow_\$ [q]$, runs $\mathcal{A}^{|\mathsf{H}_1\rangle}(z)$ until just before i-th query, measures a query input register in the computational basis and outputs the measurement outcome as x. Then,*

$$\Pr\left[\mathcal{A}^{|\mathsf{H}_1 \setminus \mathcal{S}\rangle}(z) : \mathbf{find} = \top\right] \leq 4q \Pr\left[x \leftarrow \mathcal{B}^{|\mathsf{H}_1\rangle, \mathcal{A}}(z) : x \in \mathcal{S}\right].$$

In particular, if $\Pr[x \in \mathcal{S}] \leq \epsilon$ for each $x \in \mathcal{S}$ (conditioned on z, on other oracles, and on other outputs of H_1), then $\Pr\left[\mathcal{A}^{|\mathsf{H}_1 \setminus \mathcal{S}\rangle}(z) : \mathbf{find} = \top\right] \leq 4q\epsilon$.

3 ML-DSA

In this section, we review ML-DSA and introduce the simplified ML-DSA for technical reasons.

Before going into the details, we give intuition behind ML-DSA by following its cast, the Bai-Galbraith signature [BG14]. The secret key is two short vectors $\boldsymbol{s}_1 \in R_q^\ell$ and $\boldsymbol{s}_2 \in R_q^k$. The public key consists of a random matrix $\boldsymbol{A} \in R_q^{k \times \ell}$ and a vector $\boldsymbol{t} = \boldsymbol{A}\boldsymbol{s}_1 + \boldsymbol{s}_2$. The commitment is $\boldsymbol{w} = \boldsymbol{A}\boldsymbol{y}$, where $\boldsymbol{y} \in R_q^\ell$ is a short vector. The challenge is a short, low-weight polynomial $c \in R_q$. The response is $\boldsymbol{z} = \boldsymbol{y} + c\boldsymbol{s}_1$. Intuitively speaking, the verification algorithm checks if $\boldsymbol{z}$ is short and the higher order of $\boldsymbol{A}\boldsymbol{z} - c\boldsymbol{t}$ matches those of $\boldsymbol{w}$. To prevent the leakage of the secrets $(\boldsymbol{s}_1, \boldsymbol{s}_2)$, the signing procedure requires the aborting condition related to the length of $\boldsymbol{z} = \boldsymbol{y} + c\boldsymbol{s}_1$ and low-order bits of $\boldsymbol{w} - c\boldsymbol{s}_2$.

The concrete algorithms of ML-DSA, denoted by ML-DSA, are depicted in Figs. 4 to 6 (for the missing definitions, see FIPS204[Nat24a]),[10] where $\mathsf{H}(\mathsf{str}, \ell) = \mathsf{SHAKE256}(\mathsf{str}, 8\ell)$ if str is a bit string and $\mathsf{SHAKE256}(\mathsf{BytesToBits}(\mathsf{str}), 8\ell)$ if str is a byte string. ML-DSA generally follows the hedged Fiat-Shamir with aborts paradigm, employing a *hedged extractor* to derive randomness from the message/nonce. In the hedged ML-DSA, the nonce rnd is randomly chosen from $\mathbb{B}^{32}$. [11] The following differences exist between the signing algorithms of ML-DSA (Fig. 5) and the *typical* hedged Fiat-Shamir with aborts (see Appendix B).

- In the signature generation, ML-DSA derives a message representative μ from the encoded message M' and uses μ as the input for subsequent functions, which is required to obtain the BUFF properties defined in [CDF+21].
- Instead of generating per-loop randomness directly via the hedged extractor, ML-DSA first generates a seed ρ'' using the hedged extractor (Line 9 of Fig. 5) and expands it through a function called *expand mask* (Line 13 of Fig. 5). Here, the hedged extractor is implemented as $\mathsf{H}(K \| rnd \| \mu, 64)$, where K is a part of a private key.

[10] For simplicity, we define Alg_internal as Alg for $\mathsf{Alg} \in \{\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify}\}$.
[11] The nonce rnd is fixed to $\mathbf{0} = (0, \ldots, 0) \in \mathbb{B}^{32}$ in deterministic ML-DSA.

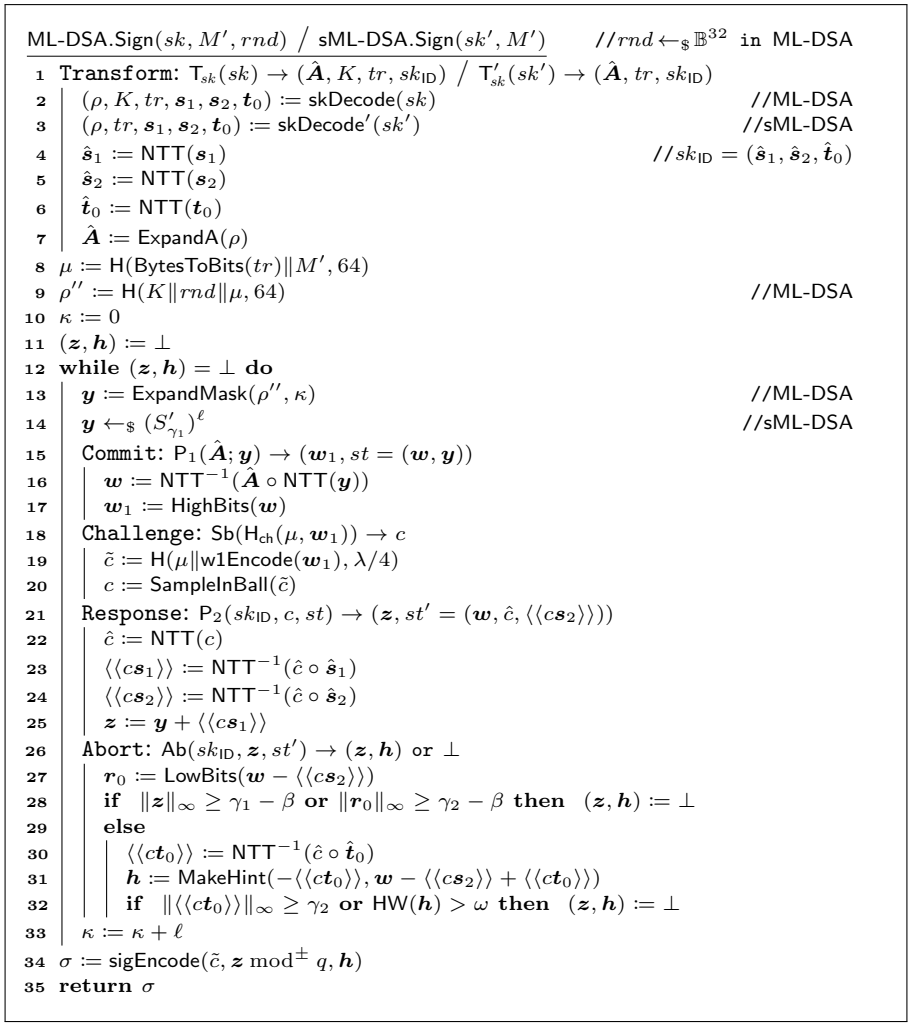

Fig. 5: Signature generation of ML-DSA and simplified ML-DSA

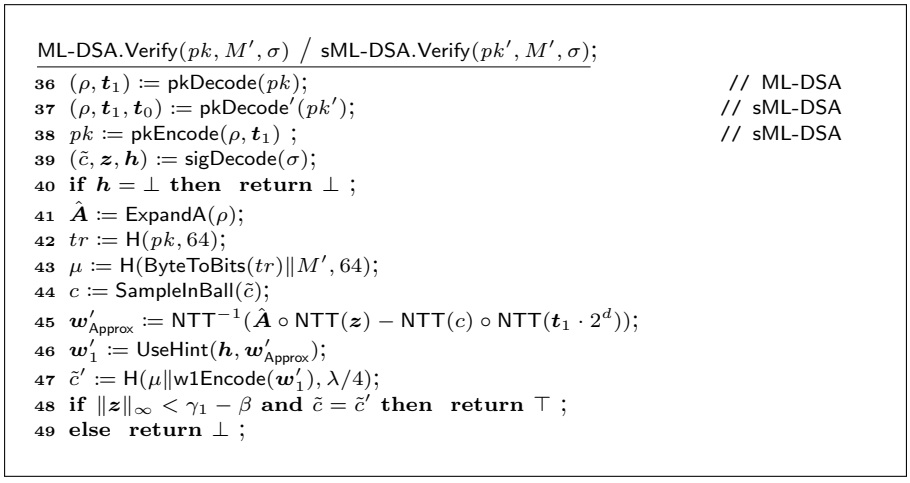

Fig. 6: Signature verification of ML-DSA and simplified ML-DSA

Note that we assume that the number of loop iterations is bounded by $B \in [814, 2^{16}/\ell]$, which makes the error at most 2^{-256} [Nat24a, Appendix C].[12]

[12] The specification assumes that a counter κ is represented by 2 bytes, which is the reason of the number $2^{16}/\ell$.

In addition, we define the simplified ML-DSA, denoted by sML-DSA, as in Figs. 4 to 6. We note that the signing algorithm samples per-loop randomness uniformly at random instead of ExpandMask, and the public key is not compressed for two purposes: First, we need to consider the intermediate security and the EUF-NMA security of *non-hedged* ML-DSA as [AOTZ20] considered those of the non-hedged signature scheme. Second, the public-key compression in ML-DSA prevents the simulation of signatures using snaHVZK, as discussed in [BBD+23]. Note that we denote the public/private keys of sML-DSA by (pk', sk') and $\mathsf{pkEncode}'$, $\mathsf{skEncode}'$, $\mathsf{pkDecode}'$, and $\mathsf{skDecode}'$ represent the encoding and decoding functions corresponding (pk', sk').

4 Security Definitions for ML-DSA

We extend the security definition of hedged Fiat-Shamir against fault-injection attacks, as defined by Aranha et al. [AOTZ20, Sec. 4], to the context of ML-DSA. This section begins by introducing the necessary notations and then defines the security notions for ML-DSA and its simplified variant.

First, we introduce the following notations to abstract the processes and variables used in ML-DSA to streamline the proofs.

$sk_{\mathsf{ID}} = (\hat{\boldsymbol{s}}_1, \hat{\boldsymbol{s}}_2, \hat{\boldsymbol{t}}_0)$: part of sk used for the underlying ID scheme ID.
$\mathsf{T}_{sk}(sk)/\mathsf{T}'_{sk}(sk')$ (Lines 2 to 7 in Fig. 5): function to transform a private key of ML-DSA (T_{sk}) or sML-DSA (T'_{sk}) to the form used to generate signatures.
$\mathsf{P}_1(\hat{\boldsymbol{A}}; \boldsymbol{y})$ (Lines 16 and 17 in Fig. 5): function to make a commitment $\boldsymbol{w}_1$ and output a state $st = (\boldsymbol{w}, \boldsymbol{y})$.
$\mathsf{P}_2(sk_{\mathsf{ID}}, c, st)$ (Lines 22 to 25 in Fig. 5): function to compute a response $\boldsymbol{z}$ and output a state $st' = (\boldsymbol{w}, \hat{c}, \langle\langle c\boldsymbol{s}_2 \rangle\rangle)$.
$\mathsf{Ab}(sk_{\mathsf{ID}}, \boldsymbol{z}, st')$ (Lines 27 to 32 in Fig. 5): function to abort an invalid response. Regarding the output, if the algorithm does not abort, it outputs $(\boldsymbol{z}, \boldsymbol{h})$; if it aborts, it outputs $\perp$.
$\mathsf{H}_\mu(tr, M') = \mathsf{H}(\mathsf{BytesToBits}(tr) \| M', 64)$: hashing to generate a message representative $\mu \in \mathbb{B}^{64}$. H_μ is modeled as a random function drawn from $\mathsf{Func}(\mathbb{B}^{64} \times \{0,1\}^*, \mathbb{B}^{64})$.
$\mathsf{H}_{\mathsf{he}}(K, rnd, \mu) = \mathsf{H}(K \| rnd \| \mu, 64)$: hashing called hedged extractor, which generates a private random seed $\rho'' \in \mathbb{B}^{64}$. H_{he} is modeled as a random function drawn from $\mathsf{Func}(\mathbb{B}^{32} \times \mathbb{B}^{32} \times \mathbb{B}^{64}, \mathbb{B}^{64})$.
$\mathsf{H}_{\mathsf{em}}(\rho'', \kappa) = \mathsf{BitUnpack}(\mathsf{H}(\rho'' \| \mathsf{IntegerToBytes}(\kappa, 2), 32(\log_2(\gamma_1)+1)), \gamma_1 - 1, \gamma_1)$[13]: hashing that generates $\boldsymbol{y} \in S'_{\gamma_1}$ and forms ExpandMask. Specifically, $\boldsymbol{y} = (y_1, \ldots, y_\ell) \coloneqq \mathsf{ExpandMask}(\rho'', \kappa)$ in Line 13 in Fig. 5 is implemented as $y_j = \mathsf{H}_{\mathsf{em}}(\rho'', \kappa + j - 1)$ (see [Nat24a, Algorithm 34]). H_{em} is modeled as a random function drawn from $\mathsf{Func}(\mathbb{B}^{64} \times [0, 2^{16} - 1], S'_{\gamma_1})$.
$\mathsf{H}_{\mathsf{ch}}(\mu, \boldsymbol{w}_1) = \mathsf{H}(\mu \| \mathsf{w1Encode}(\boldsymbol{w}_1), \lambda/4)$: hashing called challenge hash, which generates a challenge $\tilde{c} \in \mathcal{C}$, where λ is a parameter defining the collision resistance. H_{ch} is modeled as a random function drawn from $\mathsf{Func}(\mathbb{B}^{64} \times R_q^k, \mathbb{B}^{\lambda/4})$.
$\mathsf{Sb}(\tilde{c}) = \mathsf{SampleInBall}(\tilde{c})$: function to derive a challenge $c \in B_\tau$ from the output of the challenge hash, where B_τ is the set of polynomials in R whose coefficients are in $\{-1, 0, 1\}$ and Hamming weight is τ.
$\mathsf{H}_{\mathsf{es}}(\xi, k, \ell) = \mathsf{H}(\xi \| \mathsf{IntegerToBytes}(k, 1) \| \mathsf{IntegerToBytes}(\ell, 1), 128)$: hashing called expand seed used in the key generation, which generates public seed $\rho \in \mathbb{B}^{32}$, private seed $\rho' \in \mathbb{B}^{64}$, and a key $K \in \mathbb{B}^{32}$ from a seed $\xi \in \mathbb{B}^{32}$. H_{es} is modeled as a random function drawn from $\mathsf{Func}(\mathbb{B}^{32} \times \mathbb{B} \times \mathbb{B}, \mathbb{B}^{32} \times \mathbb{B}^{64} \times \mathbb{B}^{32})$.

Note that we define the abort procedure, specific to Fiat-Shamir with aborts, as a standalone function Ab rather than embedding it into the response P_2. This separation enables a focused analysis of the abort procedure, as its condition check plays a critical role in preventing the leakage of $\boldsymbol{s}_1$ and $\boldsymbol{s}_2$. Similarly, we separate challenge generation into the challenge hash H_{ch} and the challenge sampling function Sb, enabling separate analyses of their fault resilience.

Subsequently, we define the security of ML-DSA with fault injection. As a preliminary step, we formalize fault functions. Let $\Phi = \{\mathsf{set\_bit}_{\mathcal{T}}, \mathsf{flip\_bit}_{\mathcal{I}}, \mathsf{id}\}$[14] be a set of tampering and identity functions defined as:

$\mathsf{set\_bit}_{\mathcal{T}}(x)$: sets i-th bit of x to b for each $(i, b) \in \mathcal{T}$.
$\mathsf{flip\_bit}_{\mathcal{I}}(x)$: does a logical negation of the i-th bit of x for each $i \in \mathcal{I}$.

[13] γ_1 is always a power of 2.
[14] We assume that $\phi(\perp) = \perp$ for any ϕ.

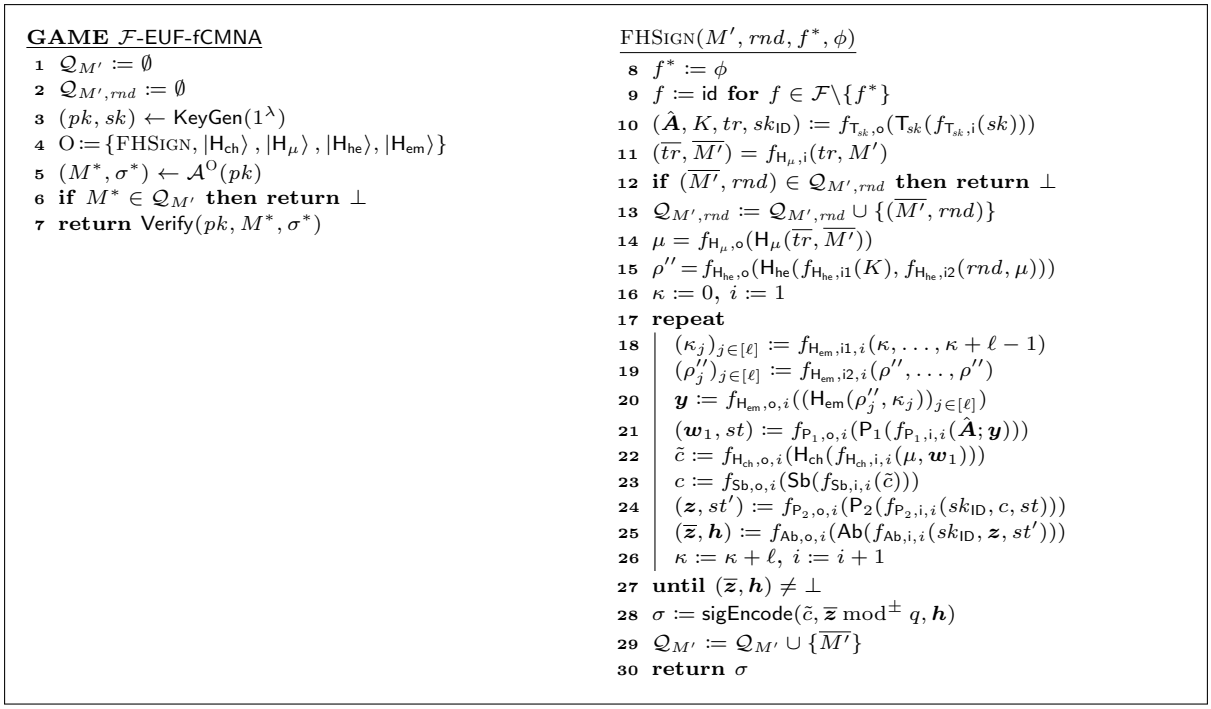

Fig. 7: $\mathcal{F}$-EUF-fCMNA game for ML-DSA

$\mathsf{id}(x)$: outputs x as it is.

Generalizing the model of [AOTZ20] that only assumes single-bit faults, we assume that the adversary can cause faults in at most t bits, while we keep the restriction that one fault-injection function per one signing query; that is, an adversary can query with a fault function $\phi \in \Phi$.

Then, we define the EUF-fCMNA security of ML-DSA.

Definition 6 (EUF-fCMNA for ML-DSA). *Using a game given in Fig. 7, we define an advantage function of the adversary playing* EUF-fCMNA *game as*

$$\mathrm{Adv}^{\text{EUF-fCMNA}}_{\text{ML-DSA}}(\mathcal{A}) = \Pr\left[\mathcal{F}\text{-EUF-CMNA}^{\mathcal{A}} = \top\right],$$

where $\mathcal{F} \subseteq \{f_{\mathsf{T}_{sk},\mathsf{i}}, f_{\mathsf{T}_{sk},\mathsf{o}}, f_{\mathsf{H}_\mu,\mathsf{i}}, f_{\mathsf{H}_\mu,\mathsf{o}}, f_{\mathsf{H}_{\mathsf{he}},\mathsf{i}1}, f_{\mathsf{H}_{\mathsf{he}},\mathsf{i}2}, f_{\mathsf{H}_{\mathsf{he}},\mathsf{o}}\} \cup \{f_{\mathsf{H}_{\mathsf{em}},\mathsf{i}1,i}, f_{\mathsf{H}_{\mathsf{em}},\mathsf{i}2,i}, f_{\mathsf{H}_{\mathsf{em}},\mathsf{o},i}, f_{\mathsf{P}_1,\mathsf{i},i}, f_{\mathsf{P}_1,\mathsf{o},i}, f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}, f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}, f_{\mathsf{Sb},\mathsf{i},i}, f_{\mathsf{Sb},\mathsf{o},i}, f_{\mathsf{P}_2,\mathsf{i},i}, f_{\mathsf{P}_2,\mathsf{o},i}, f_{\mathsf{Ab},\mathsf{i},i}, f_{\mathsf{Ab},\mathsf{o},i}\}_{i\in[B]}$[15] *and* $\mathcal{A}$ *is allowed to assign any* $\phi \in \Phi$ *to* $f^* \in \mathcal{F}$ *within* FHSign. *We say* ML-DSA *is* $\mathcal{F}$-EUF-fCMNA*-secure if its corresponding advantage is negligible for any efficient adversary in the security parameter.*

Note that the original security notion of $\mathcal{F}$-EUF-fCMNA-security in [AOTZ20, Definition 6] does not address the situation where the adversary submits multiple queries of the same message and nonce; however, the security proof of the hedged Fiat-Shamir in [AOTZ20, Lemma 4] explicitly prohibits such queries in its statement. To make the security notion more precise, we modify it so that $\mathcal{Q}_{M',rnd}$ stores queried message/nonce pairs, thereby preventing duplicate submissions. Moreover, we assume that $\mathcal{Q}_{M',rnd}$ records possibly tampered messages $\overline{M'}$ rather than the original M'. This adjustment prevents a trivial attack in which two signatures are generated for the same message/nonce pair by injecting a fault into M' via $f_{\mathsf{H}_\mu,\mathsf{i}}$. Moreover, we do not consider faults on the input or output of the encoding function defined in [AOTZ20, Definition 6], as proving resilience against such faults is trivial.

In addition, we introduce two security definitions of the simplified ML-DSA.

[15] To specify the location where a fault is injected, we use the function's symbol, its input/output (denoted by i and o, respectively), and the loop index i if the function is inside a loop.

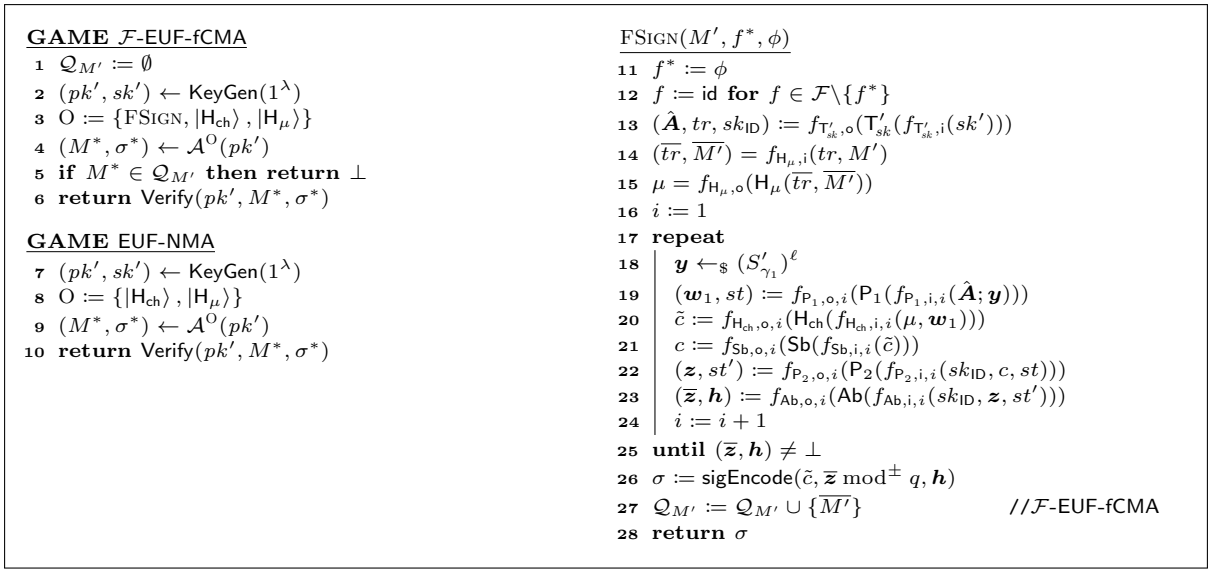

Fig. 8: $\mathcal{F}$-EUF-fCMA and EUF-NMA games for simplified ML-DSA

Definition 7 (EUF-fCMA and EUF-NMA for Simplified ML-DSA). *Using games given in Fig. 8, we define advantage functions of adversaries playing* EUF-fCMA *and* EUF-NMA *games as*

$$\mathrm{Adv}^{\text{EUF-fCMA}}_{\text{sML-DSA}}(\mathcal{A}) = \Pr\left[\mathcal{F}\text{-EUF-CMA}^{\mathcal{A}} = \top\right]$$
$$\textit{and} \quad \mathrm{Adv}^{\text{EUF-NMA}}_{\text{sML-DSA}}(\mathcal{A}) = \Pr\left[\text{EUF-NMA}^{\mathcal{A}} = \top\right],$$

where $\mathcal{F} \subseteq \{f_{\mathsf{T}'_{sk},\mathsf{i}}, f_{\mathsf{T}'_{sk},\mathsf{o}}, f_{\mathsf{H}_\mu,\mathsf{i}}, f_{\mathsf{H}_\mu,\mathsf{o}}\} \cup \{f_{\mathsf{P}_1,\mathsf{i},i}, f_{\mathsf{P}_1,\mathsf{o},i}, f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}, f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}, f_{\mathsf{Sb},\mathsf{i},i}, f_{\mathsf{Sb},\mathsf{o},i}, f_{\mathsf{P}_2,\mathsf{i},i}, f_{\mathsf{P}_2,\mathsf{o},i}, f_{\mathsf{Ab},\mathsf{i},i}, f_{\mathsf{Ab},\mathsf{o},i}\}_{i\in[B]}$ *and* $\mathcal{A}$ *is allowed to assign any* $\phi \in \Phi$ *to* $f^* \in \mathcal{F}$ *within* FSign. *We say* sML-DSA *is* $\mathcal{F}$-EUF-fCMA*-secure if its corresponding advantage is negligible for any efficient adversary in the security parameter.*

5 Security Proofs

The security proof is divided into two parts. The first part reduces the EUF-NMA security to the $\mathcal{F}$-EUF-fCMA security in the simplified ML-DSA. The second part reduces the $\mathcal{F}$-EUF-fCMA security of the simplified ML-DSA to the $\mathcal{F}'$-EUF-fCMNA security of ML-DSA. These two parts are proven in Sections 5.3 and 5.4, respectively. Before presenting these proofs, Section 5.1 introduces auxiliary proof techniques in the QROM and Section 5.2 shows that the underlying ID scheme of ML-DSA is snaHVZK, which serves as a foundation for the subsequent security proofs.

5.1 Auxiliary Proof Techniques in QROM

We introduce the following two lemmas used in the security proof (See the proofs in Appendix C.)

Lemma 4 (O2H with Adaptive Reprogramming and Positioning). *Let* $\mathsf{H}_0, \mathsf{H}_1 \colon \mathcal{X} \to \mathcal{Y}$ *be functions that are reprogrammed depending on the results of classical queries to an oracle* O *(*H_0 *and* H_1 *may be reprogrammed differently). Assume that* $\mathsf{H}_0(x) = \mathsf{H}_1(x)$ *for all* $x \notin \mathcal{S}_i$*, where* $\mathcal{S}_i$ *is defined based on the results of queries to* O *up to the* i*-th query. Let* z *be a random bitstring. (*H_0*,* H_1*,* z *may have arbitrary joint distribution.) Let* $\mathcal{A}$ *be a quantum algorithm with* q *quantum queries to* H_0 *or* H_1 *and some classical queries to* O*. Let* $\mathcal{B}$ *be a quantum algorithm that, given access to the oracles* H_0 *and* O *and the adversary* $\mathcal{A}$*, performs the following steps on input* z*: picks* $i \leftarrow_{\$} [q]$*, runs* $\mathcal{A}$ *until the* i*-th query, measures the query input register in the computational basis, and outputs the measurement outcome* x *along with the index* i *along. Then, we have*

$$\left|\Pr\left[\mathcal{A}^{|\mathsf{H}_0\rangle,\mathrm{O}}(z)=1\right] - \Pr\left[\mathcal{A}^{|\mathsf{H}_1\rangle,\mathrm{O}}(z)=1\right]\right| \leq 2q\sqrt{\Pr\left[(i,x) \leftarrow \mathcal{B}^{|\mathsf{H}_0\rangle,\mathrm{O},\mathcal{A}}(z) : x \in \mathcal{S}_i\right]}.$$

$\underline{\mathsf{Trans}(sk', c)}$

$(\rho, tr, \boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{t}_0) \coloneqq \mathsf{skDecode}'(sk')$
$\boldsymbol{A} \coloneqq \mathsf{ExpandA}'(\rho)$
$\boldsymbol{y} \leftarrow_{\$} (S'_{\gamma_1})^{\ell}$
$\boldsymbol{w} \coloneqq \boldsymbol{A}\boldsymbol{y}$
$\boldsymbol{w}_1 \coloneqq \mathsf{HighBits}(\boldsymbol{w})$
$\boldsymbol{z} \coloneqq \boldsymbol{y} + c\boldsymbol{s}_1$
$\boldsymbol{r}_0 \coloneqq \mathsf{LowBits}(\boldsymbol{w} - c\boldsymbol{s}_2)$
if $\|\boldsymbol{z}\|_\infty \geq \gamma_1 - \beta$ **then**
  $(\boldsymbol{z}, \boldsymbol{h}) \coloneqq \perp$
else if $\|\boldsymbol{r}_0\|_\infty \geq \gamma_2 - \beta$ **then**
  $(\boldsymbol{z}, \boldsymbol{h}) \coloneqq \perp$
else
  $\boldsymbol{h} \coloneqq \mathsf{MakeHint}(-c\boldsymbol{t}_0, \boldsymbol{w} - c\boldsymbol{s}_2 + c\boldsymbol{t}_0)$
  if $\|c\boldsymbol{t}_0\|_\infty \geq \gamma_2$ **or** $\mathsf{HW}(\boldsymbol{h}) > \omega$ **then**
    $(\boldsymbol{z}, \boldsymbol{h}) \coloneqq \perp$
if $(\boldsymbol{z}, \boldsymbol{h}) = \perp$ **then return** $\perp$
return $(\boldsymbol{w}_1, \boldsymbol{z}, \boldsymbol{h})$

$\underline{\mathsf{Sim}(pk', c)}$

return $\perp$
  with probability $1 - \left(\frac{2\gamma_1 - 2\beta + 1}{2\gamma_1}\right)^{256\ell}$
$(\rho, \boldsymbol{t}_1, \boldsymbol{t}_0) \coloneqq \mathsf{pkDecode}'(pk')$
$\boldsymbol{t} \coloneqq 2^d \cdot \boldsymbol{t}_1 + \boldsymbol{t}_0$
$\boldsymbol{A} \coloneqq \mathsf{ExpandA}'(\rho)$
$\boldsymbol{z} \leftarrow_{\$} S^{\ell}_{\gamma_1 - \beta - 1}$
$\boldsymbol{w}_1 \coloneqq \mathsf{HighBits}(\boldsymbol{A}\boldsymbol{z} - c\boldsymbol{t})$
$\boldsymbol{r}_0 \coloneqq \mathsf{LowBits}(\boldsymbol{A}\boldsymbol{z} - c\boldsymbol{t})$
if $\|\boldsymbol{r}_0\|_\infty \geq \gamma_2 - \beta$ **then**
  $(\boldsymbol{z}, \boldsymbol{h}) \coloneqq \perp$
else
  $\boldsymbol{h} \coloneqq \mathsf{MakeHint}(-c\boldsymbol{t}_0, \boldsymbol{A}\boldsymbol{z} - c\boldsymbol{t} + c\boldsymbol{t}_0)$
  if $\|c\boldsymbol{t}_0\|_\infty \geq \gamma_2$ **or** $\mathsf{HW}(\boldsymbol{h}) > \omega$ **then**
    $(\boldsymbol{z}, \boldsymbol{h}) \coloneqq \perp$
if $(\boldsymbol{z}, \boldsymbol{h}) = \perp$ **then return** $\perp$
return $(\boldsymbol{w}_1, \boldsymbol{z}, \boldsymbol{h})$

Fig. 9: Honest generation of transcript and snaHVZK simulator of ML-DSA

Lemma 5 (Collison Resistance with Fault Injection against Random Function). *Let* $\mathsf{H}\colon \mathcal{X} \to \mathcal{Y}$ *be a random function. Then, any quantum algorithm that makes* q *quantum queries to* H *outputs a pair such that* $\mathsf{H}(x) = \phi(\mathsf{H}(x'))$ *with probability at most* $632 \cdot 2^t \binom{\log_2(|\mathcal{Y}|)+1}{t} (q+1)^3/|\mathcal{Y}|$*, and a pair such that* $\phi(\mathsf{H}(x)) = \phi'(\mathsf{H}(x'))$ *with probability at most* $632 \cdot 2^{2t} \binom{\log_2(|\mathcal{Y}|)+1}{2t} (q+1)^3/|\mathcal{Y}|$*, where* $\phi, \phi' \in \Phi$*.*

5.2 Special No-Abort HVZK of Simplified ML-DSA

Let Trans and Sim denote honest generation and simulation of transcripts in the underlying ID scheme ID of the simplified ML-DSA, as defined in Fig. 9, where we define $\mathsf{ExpandA}'$ as the expansion of the private seed ρ to non-NTT representation of $\boldsymbol{A}$. By incorporating Sim, ID becomes snaHVZK. For the proof, see Appendix C.3.

Lemma 6 (Special No-Abort HVZK of Simplified ML-DSA). *Let* ID *be the underlying ID scheme of the simplified ML-DSA. Then,* ID *is snaHVZK.*

5.3 EUF-NMA to $\mathcal{F}$-EUF-fCMA

We show the reduction of $\mathcal{F}$-EUF-fCMA to EUF-NMA of the simplified ML-DSA $\mathsf{sML\text{-}DSA}$.

Theorem 1 (EUF-NMA $\Rightarrow$ $\mathcal{F}$-EUF-fCMA). *Assume the underlying* ID *of* $\mathsf{sML\text{-}DSA}$ *has* (α, δ)*-min-entropy and correctness error* ε *and is snaHVZK, and the bitlength of commitment is* $\mathsf{bitlen}(\boldsymbol{w}_1)$*. For any* $\mathcal{F}$-EUF-fCMA *adversary* $\mathcal{A}_{\mathsf{cma}}$ *of* $\mathsf{sML\text{-}DSA}$ *issuing at most* $q_{\mathsf{H}_{\mathsf{ch}}}$ *and* q_{H_μ} *quantum queries to* H_{ch} *and* H_μ*, respectively, and* q_{S} *classical queries to* FSign*, there exists an* EUF-NMA *adversary* $\mathcal{A}_{\mathsf{nma}}$ *of* $\mathsf{sML\text{-}DSA}$ *such that*

$$\mathrm{Adv}^{\text{EUF-fCMA}}_{\mathsf{sML\text{-}DSA}}(\mathcal{A}_{\mathsf{cma}}) \leq \mathrm{Adv}^{\text{EUF-NMA}}_{\mathsf{sML\text{-}DSA}}(\mathcal{A}_{\mathsf{nma}}) + \frac{3}{2} q'_{\mathsf{S}} \sqrt{\frac{q_{\mathsf{H}_{\mathsf{ch}}} + q'_{\mathsf{S}} + 1}{2^{\alpha - t}}} + 2 q_{\mathsf{H}_{\mathsf{ch}}} \sqrt{\frac{q'_{\mathsf{S}}}{2^{\alpha - t}}} + \frac{\binom{513}{t}(q_{\mathsf{H}_\mu} + 1)^3}{2^{502 - t}} + \frac{\sum_{i \in [2t+1]} \binom{\mathsf{bitlen}(\boldsymbol{w}_1)}{i-1} {q_{\mathsf{S}}}^2}{2^{\alpha}} + 4\delta,$$

where $\mathcal{F} = \{f_{\mathsf{H}_\mu,\mathsf{i}}, f_{\mathsf{H}_\mu,\mathsf{o}}\} \cup \{f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}, f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}, f_{\mathsf{Sb},\mathsf{i},i}, f_{\mathsf{Ab},\mathsf{o},i}\}_{i \in [B]}$*,* $q'_{\mathsf{S}} = \theta q_{\mathsf{S}}/(1-\varepsilon)$ *for some constant* $\theta > 1$*, and the running time of* $\mathcal{A}_{\mathsf{nma}}$ *is about that of* $\mathcal{A}_{\mathsf{cma}}$*.*

Proof. We show the security bound using the games shown in Fig. 10. Note that q'_{S} denotes the bound on the total number of H_{ch} calls across all signing queries. By setting $q'_{\mathsf{S}} = \theta q_{\mathsf{S}}/(1-\varepsilon)$, the probability that this bound is exceeded becomes negligible, following the same reasoning as the proof of [KX24a, Theorem 1].

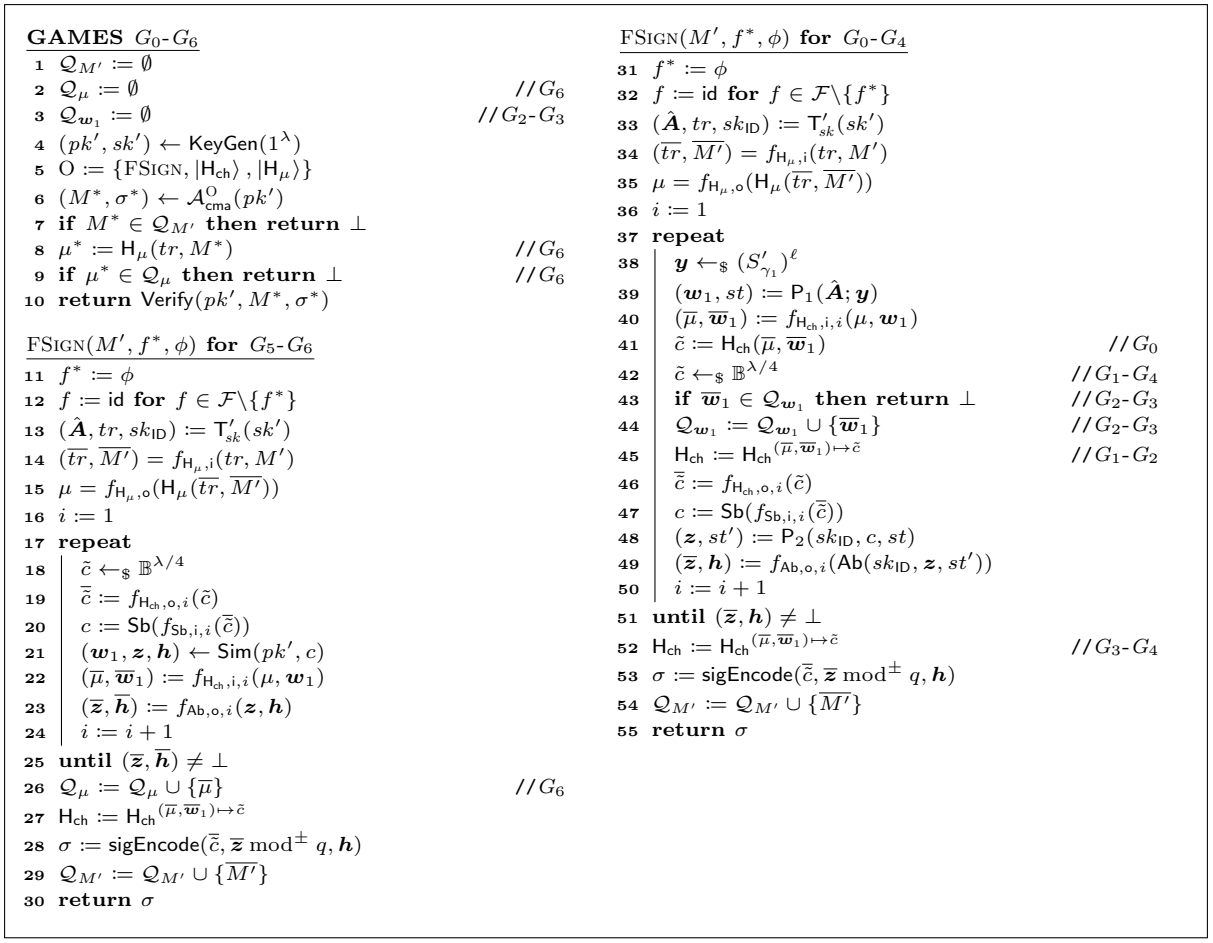

Fig. 10: Games for EUF-NMA $\Rightarrow$ $\mathcal{F}$-EUF-fCMA

GAME G_0: This is the original $\mathcal{F}$-EUF-fCMA game and we have

$$\Pr[W_0] = \mathrm{Adv}^{\mathcal{F}\text{-EUF-fCMA}}_{\mathsf{ML\text{-}DSA}}(\mathcal{A}_{\mathsf{cma}}).$$

GAME G_1: The signing oracle FSIGN uniformly chooses $\tilde{c}$ and adaptively reprograms H_{ch} as $\mathsf{H}_{\mathsf{ch}} \coloneqq \mathsf{H}_{\mathsf{ch}}^{(\overline{\mu}, \overline{\boldsymbol{w}}_1) \mapsto \tilde{c}}$. Due to this adaptive reprogramming, simulation by Sim becomes feasible in the next step.

Lemma 7. *Suppose that* ID *has* (α, δ)*-min-entropy and* q'_{S} *bounds the number of queries to* H_{ch} *across all signing queries. Then, we have*

$$|\Pr[W_0] - \Pr[W_1]| \leq \frac{3}{2} q'_{\mathsf{S}} \sqrt{\frac{q_{\mathsf{H}_{\mathsf{ch}}} + q'_{\mathsf{S}} + 1}{2^{\alpha - t}}} + \delta.$$

Proof. First, we eliminate the bad event where the key generation algorithm produces a key pair that results in the commitment having min-entropy less than α. This elimination introduces a difference of δ in the bound. Since the same process is performed in the following lemmas, we omit the descriptions.

Then, assuming that the commitment has a min-entropy α, we prove this lemma using tight adaptive reprogramming (see Lemma 1). The AR adversary $\mathcal{B}_{\mathsf{ar}}$ to distinguish H_0 and H_1 (see Fig. 3) can simulate G_0 and G_1, where H_{ch} is a random function reprogrammed in the AR game. To simulate FSIGN, $\mathcal{B}_{\mathsf{ar}}$ makes reprogramming queries considering faults on $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}$ with $D_{\mu,\phi}$, a distribution of $((\overline{\mu}, \overline{\boldsymbol{w}}_1), st)$ generated by $(\boldsymbol{w}_1, st) \coloneqq \mathsf{P}_1(\hat{\boldsymbol{A}}; \boldsymbol{y})$ and $(\overline{\mu}, \overline{\boldsymbol{w}}_1) \coloneqq \phi(\mu, \boldsymbol{w}_1)$. If $\mathcal{B}_{\mathsf{ar}}$ plays AR_0, $\mathsf{H}_{\mathsf{ch}} = \mathsf{H}_0$ is not reprogrammed; therefore, it simulates G_0; otherwise, $\mathsf{H}_{\mathsf{ch}} = \mathsf{H}_1$ is reprogrammed and $\mathcal{B}_{\mathsf{ar}}$ simulates

G_1. Hence. we have $|\Pr[W_0] - \Pr[W_1]| \leq \mathrm{Adv}^{\mathrm{AR}}_{\mathsf{H}}(\mathcal{B}_{\mathsf{ar}}) + \delta$. Since $\phi \in \Phi$ is applied, the min-entropy of $\overline{\boldsymbol{w}}_1$ may be reduced to $\alpha - t$. Since the number of queries to H_{ch} across all signing queries is at most q'_{S}, the number of reprogramming queries is also at most q'_{S}. Thus, from Lemma 1, we obtain the bound.

GAME G_2: We introduce $\mathcal{Q}_{\boldsymbol{w}_1}$ to store tampered commitments $\overline{\boldsymbol{w}}_1$, ensuring that FSIGN returns $\perp$ if a new $\overline{\boldsymbol{w}}_1$ already exists in $\mathcal{Q}_{\boldsymbol{w}_1}$. This prevents H_{ch} from being reprogrammed for the same commitment more than once.

Lemma 8. *Suppose that* ID *has* (α, δ)*-min-entropy and* q'_{S} *bounds the number of queries to* H_{ch} *across all signing queries. Then, we have*

$$|\Pr[W_1] - \Pr[W_2]| \leq \frac{\sum_{i\in[2t+1]}\binom{\mathsf{bitlen}(\boldsymbol{w}_1)}{i-1} {q_{\mathsf{S}}}^2}{2^{\alpha+1}} + \delta.$$

Proof. We can bound $|\Pr[W_1] - \Pr[W_2]|$ by the collision probability of selecting $\boldsymbol{w}_1$ while allowing differences of up to $2t$ bits. If any $\boldsymbol{w}_1$ is generated such that each instance differs by at least $2t+1$ bits, the fault-injected $\phi(\boldsymbol{w}_1)$ will not belong to $\mathcal{Q}_{\boldsymbol{w}_1}$. Consequently, in such cases, G_1 and G_2 become indistinguishable. Thus, $|\Pr[W_1] - \Pr[W_2]|$ is bounded by the collision probability allowing differences of up to $2t$ bits.

Then, we derive a bound on the collision probability of $\boldsymbol{w}_1$ when differences of up to $2t$ bits are allowed. There are $\sum_{i\in[2t+1]}\binom{\mathsf{bitlen}(\boldsymbol{w}_1)}{i-1}$ possible combinations for taking up to $2t$ bits differences. Multiplying this count by $\binom{q'_{\mathsf{S}}}{2}2^{-\alpha}$ and simplifying it, we obtain the bound.

GAME G_3: We cancel the reprogramming for aborted transcripts, ensuring that reprogramming is performed only for no-aborting transcripts. Due to this cancellation, we do not need to simulate commitments for aborted transcripts, allowing us to use snaHVZK to simulate FSIGN.

Lemma 9. *Suppose that* ID *has* (α, δ)*-min-entropy and* q'_{S} *bounds the number of queries to* H_{ch} *across all signing queries. Then, we have*

$$|\Pr[W_2] - \Pr[W_3]| \leq 2q_{\mathsf{H}_{\mathsf{ch}}}\sqrt{\frac{q'_{\mathsf{S}}}{2^{\alpha-t}}} + \delta.$$

Proof. We use O2H with adaptive reprogramming and positioning given in Lemma 4 to bound $|\Pr[W_2] - \Pr[W_3]|$, where we set $\mathrm{O} = \mathrm{FSIGN}$ and define $\mathcal{S}_j$ as the set of inputs $(\overline{\mu}, \overline{\boldsymbol{w}}_1)$ used for reprogramming in G_2 but not in G_3 up to the j-th query to H_{ch}. Let $\mathcal{B}_{\mathsf{o2h}}$ be an adversary who runs $\mathcal{A}_{\mathsf{cma}}$ in G_3 and finds an element in $\mathcal{S}_j$. Choosing $j \leftarrow_{\$} [q_{\mathsf{H}_{\mathsf{ch}}}]$, $\mathcal{B}_{\mathsf{o2h}}$ measures the query input register of $\mathcal{A}_{\mathsf{cma}}$ and returns the result. The adversary $\mathcal{B}_{\mathsf{o2h}}$ has no information on $\mathcal{S}_j$, and the number of distinct commitments among the elements in $\mathcal{S}_j$ is at most q'_{S}. Since the adversary can degrade the min-entropy of the commitments by t-bit by injecting faults, the probability of finding an element of $\mathcal{S}_j$ is at most $q'_{\mathsf{S}} \cdot 2^{-\alpha+t}$. Note that, due to the last modification, we do not need to consider the case where $\overline{\boldsymbol{w}}_1$ used for reprogramming in Line 52 of Fig. 10 is in $\mathcal{S}_j$. Therefore, we can derive the bound using Lemma 3.

GAME G_4: We remove $\mathcal{Q}_{\boldsymbol{w}_1}$ from the game. We have the same bound as Lemma 8.

GAME G_5: FSIGN uses Sim to simulate $(\boldsymbol{w}_1, (\boldsymbol{z}, \boldsymbol{h}))$, which enables the simulation of FSIGN by the EUF-NMA adversary.

Lemma 10. *Suppose that* ID *is snaHVZK. Then, we have* $\Pr[W_4] = \Pr[W_5]$.

Proof. To demonstrate that the adversary's view remains unchanged, we show that the values generated at the points where G_4 and G_5 differ follow the same distributions. Since ID is snaHVZK, the honestly generated $(\boldsymbol{w}_1, (\boldsymbol{z}, \boldsymbol{h}))$ and the simulated one are identically distributed. Moreover, the input/output table of H_{ch} follows the same distribution in G_4 and G_5 because it is adaptively reprogrammed for randomly chosen $\tilde{c}$ after $\overline{\boldsymbol{w}}_1$ is generated. Therefore, the adversary's view remains unchanged between G_4 and G_5.

$\mathcal{A}_{\mathsf{nma}}^{|\hat{\mathsf{H}}_{\mathsf{ch}}\rangle,|\hat{\mathsf{H}}_{\mu}\rangle}(pk')$

$\mathcal{Q}_{M'} := \emptyset$
$\mathcal{Q}_{\mu} := \emptyset$
$\mathsf{H}_{\mathsf{ch}} := \hat{\mathsf{H}}_{\mathsf{ch}}$
$tr := \mathsf{H}(\mathsf{pkComp}(pk'), 64)$
$\mathrm{O} := \{\mathrm{FSIGN}, |\mathsf{H}_{\mathsf{ch}}\rangle, |\hat{\mathsf{H}}_{\mu}\rangle\}$
$(M^*, \sigma^*) \leftarrow \mathcal{A}_{\mathsf{cma}}^{\mathrm{O}}(pk')$
if $M^* \in \mathcal{Q}_{M'}$ **then return** $\perp$
$\mu^* := \hat{\mathsf{H}}_{\mu}(tr, M^*)$
if $\mu^* \in \mathcal{Q}_{\mu}$ **then return** $\perp$
return $\mathsf{Verify}(pk', M^*, \sigma^*)$

$\mathrm{FSIGN}(M', f^*, \phi)$ simulated by $\mathcal{A}_{\mathsf{nma}}$

$f^* := \phi$
$f := \mathsf{id}$ **for** $f \in \mathcal{F} \backslash \{f^*\}$
$tr := \mathsf{H}(\mathsf{pkComp}(pk'), 64)$
$(\overline{tr}, \overline{M'}) = f_{\mathsf{H}_{\mu},\mathsf{i}}(tr, M')$
$\mu = f_{\mathsf{H}_{\mu},\mathsf{o}}(\hat{\mathsf{H}}_{\mu}(\overline{tr}, \overline{M'}))$
$i := 1$
repeat
  $\tilde{c} \leftarrow_{\$} \mathbb{B}^{\lambda/4}$
  $\overline{\tilde{c}} := f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}(\tilde{c})$
  $c := \mathsf{Sb}(f_{\mathsf{Sb},\mathsf{i},i}(\overline{\tilde{c}}))$
  $(\boldsymbol{w}_1, \boldsymbol{z}, \boldsymbol{h}) \leftarrow \mathsf{Sim}(pk', c)$
  $(\overline{\mu}, \overline{\boldsymbol{w}}_1) := f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}(\mu, \boldsymbol{w}_1)$
  $(\overline{\boldsymbol{z}}, \overline{\boldsymbol{h}}) := f_{\mathsf{Ab},\mathsf{o},i}(\boldsymbol{z}, \boldsymbol{h})$
  $i := i + 1$
until $(\overline{\boldsymbol{z}}, \overline{\boldsymbol{h}}) \neq \perp$
$\mathcal{Q}_{\mu} := \mathcal{Q}_{\mu} \cup \{\overline{\mu}\}$
$\mathsf{H}_{\mathsf{ch}} := \mathsf{H}_{\mathsf{ch}}^{(\overline{\mu}, \overline{\boldsymbol{w}}_1) \mapsto \tilde{c}}$
$\sigma := \mathsf{sigEncode}(\overline{\tilde{c}}, \overline{\boldsymbol{z}} \bmod^{\pm} q, \boldsymbol{h})$
$\mathcal{Q}_{M'} := \mathcal{Q}_{M'} \cup \{\overline{M'}\}$
return σ

Fig. 11: Simulation of the modified $\mathcal{F}$-EUF-fCMA game by EUF-NMA adversary. pkComp compresses $pk' = \mathsf{pkEncode}'(\rho, \boldsymbol{t}_0, \boldsymbol{t}_1)$ into $pk = \mathsf{pkEncode}(\rho, \boldsymbol{t}_1)$.

GAME G_6: We introduce $\mathcal{Q}_{\mu}$, which records $\overline{\mu}$, the message representative with a fault induced via either $f_{\mathsf{H}_{\mu},\mathsf{o}}$ or $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}$. If $\mu^* = \mathsf{H}_{\mu}(tr, M^*)$ for the message M^* submitted by the adversary is found in $\mathcal{Q}_{\mu}$, the game returns $\perp$.

The random function H_{ch} is reprogrammed only for $\overline{\mu} \in \mathcal{Q}_{\mu}$, and if $\mu^* \notin \mathcal{Q}_{\mu}$, the pair (M^*, σ^*) is verified using a non-reprogrammed point of H_{ch}. Therefore, the pair (M^*, σ^*) can be verified using a non-reprogrammed random function accessed by the EUF-NMA adversary.

Lemma 11. *We have*

$$|\Pr[W_5] - \Pr[W_6]| \leq \frac{\binom{513}{t}(q_{\mathsf{H}_{\mu}} + 1)^3}{2^{502-t}}.$$

Proof. When the adversary outputs M^* such that $M^* \notin \mathcal{Q}_{M'}$ in Line 7 and $\mu^* \in \mathcal{Q}_{\mu}$ in Line 9, G_5 does not return $\perp$, whereas G_6 does. Defining this event as **bad**, we have $|\Pr[W_5] - \Pr[W_6]| \leq \Pr[\mathbf{bad}]$. When **bad** occurs, a distinct input pair for H_{μ} satisfying $\mathsf{H}_{\mu}(\overline{tr}_1, \overline{M'_1}) = \phi(\mathsf{H}_{\mu}(\overline{tr}_2, \overline{M'_2}))$ has been found. Therefore, $\Pr[\mathbf{bad}]$ can be bounded by the advantage of a quantum adversary $\mathcal{B}_{\mathsf{crf}}$ against collision resistance with fault injection, which has quantum oracle access to H_{μ} and is tasked with finding $(\phi, (\overline{tr}_1, \overline{M'_1}), (\overline{tr}_2, \overline{M'_2}))$ such that $\mathsf{H}_{\mu}(\overline{tr}_1, \overline{M'_1}) = \phi(\mathsf{H}_{\mu}(\overline{tr}_2, \overline{M'_2}))$ and $(\overline{tr}_1, \overline{M'_1}) \neq (\overline{tr}_2, \overline{M'_2})$. From Lemma 5, we obtain this lemma.

Then, the EUF-NMA adversary $\mathcal{A}_{\mathsf{nma}}$ can simulate G_4 as shown in Fig. 11.

Lemma 12. *There exists an* EUF-NMA *adversary* $\mathcal{A}_{\mathsf{nma}}$ *of* sML-DSA *such that*

$$\Pr[W_6] \leq \mathrm{Adv}^{\text{EUF-NMA}}_{\text{sML-DSA}}(\mathcal{A}_{\mathsf{nma}}).$$

Proof. To demonstrate the reduction, we show that $\mathcal{A}_{\mathsf{nma}}$ can win its game whenever $\mathcal{A}_{\mathsf{cma}}$ wins in G_4. For $\mathcal{A}_{\mathsf{nma}}$ to win, it is necessary that $(\mu^*, \boldsymbol{w}_1^*)$ corresponds to (M^*, σ^*) submitted by $\mathcal{A}_{\mathsf{cma}}$ passes verification even when $\mathcal{A}_{\mathsf{nma}}$ uses its own random function $\hat{\mathsf{H}}_{\mathsf{ch}}$. Given that $\mathcal{A}_{\mathsf{cma}}$ wins, we have $\mu^* \notin \mathcal{Q}_{\mu}$, meaning that H_{ch} has not been reprogrammed for $(\mu^*, \boldsymbol{w}_1^*)$, and therefore, $\mathsf{H}_{\mathsf{ch}}(\mu^*, \boldsymbol{w}_1^*) = \hat{\mathsf{H}}_{\mathsf{ch}}(\mu^*, \boldsymbol{w}_1^*)$ holds. Consequently, $\mathcal{A}_{\mathsf{nma}}$ is able to obtain (M^*, σ^*) that passes verification with $\hat{\mathsf{H}}_{\mathsf{ch}}$, allowing it to win the game. This establishes $\Pr[W_6] \leq \mathrm{Adv}^{\text{EUF-NMA}}_{\text{sML-DSA}}(\mathcal{A}_{\mathsf{nma}})$. We note that $\mathcal{A}_{\mathsf{nma}}$ can compute $tr = \mathsf{H}(pk, 64)$ (Line 8 in Fig. 4) from $pk' = \mathsf{pkEncode}'(\rho, \boldsymbol{t}_0, \boldsymbol{t}_1)$ instead of $pk = \mathsf{pkEncode}(\rho, \boldsymbol{t}_1)$. The computation of pk from pk' is denoted by pkComp in Fig. 11.

□

Remark 1. If P_2 verifies that the challenge c is valid, i.e., $c \in B_\tau$, then Theorem 1 can be proven by including $\{f_{\mathsf{Sb},\mathsf{o},i}\}_{i\in[B]}$ in $\mathcal{F}$. This is because Sim can simulate a transcript taking $\phi(c)$ as input for any $\phi \in \Phi$ if Sim also verifies $\phi(c) \in B_\tau$.

5.4 $\mathcal{F}$-EUF-fCMA to $\mathcal{F}'$-EUF-fCMNA

We finally show the security of hedged ML-DSA, assuming one of the simplified ML-DSA.

Theorem 2 ($\mathcal{F}$-EUF-fCMA $\Rightarrow$ $\mathcal{F}'$-EUF-fCMNA). *For any $\mathcal{F}'$-EUF-fCMNA adversary $\mathcal{A}_{\mathsf{cmna}}$ of ML-DSA issuing at most $q_{\mathsf{H}_{\mathsf{ch}}}$, q_{H_μ}, $q_{\mathsf{H}_{\mathsf{he}}}$, $q_{\mathsf{H}_{\mathsf{em}}}$, and $q_{\mathsf{H}_{\mathsf{es}}}$ quantum queries to H_{ch}, H_μ, H_{he}, H_{em}, and H_{es} respectively, and q_{S} classical queries to FHSign, there exists an $\mathcal{F}$-EUF-fCMA adversary $\mathcal{A}_{\mathsf{cma}}$ of sML-DSA such that*

$$\mathrm{Adv}^{\text{EUF-fCMNA}}_{\text{ML-DSA}}(\mathcal{A}_{\mathsf{cmna}}) \le \mathrm{Adv}^{\text{EUF-fCMA}}_{\text{sML-DSA}}(\mathcal{A}_{\mathsf{cma}}) + \frac{\binom{513}{2t}(q_{\mathsf{H}_\mu}+1)^3}{2^{501-2t}} + \frac{\sqrt{\sum_{i\in[t+1]}\binom{256}{i-1}}(q_{\mathsf{H}_{\mathsf{he}}}+1)}{2^{127}} + \frac{\sqrt{\sum_{i\in[t+1]}\binom{512}{i-1}q_{\mathsf{S}}}(q_{\mathsf{H}_{\mathsf{em}}}+1)}{2^{255}} + \frac{q_{\mathsf{H}_{\mathsf{es}}}}{2^{127}} + \frac{\sum_{i\in[2t+1]}\binom{512}{i-1}{q_{\mathsf{S}}}^2}{2^{512}},$$

where $\mathcal{F}' = \mathcal{F} \cup \{f_{\mathsf{H}_{\mathsf{he}},\mathsf{i1}}, f_{\mathsf{H}_{\mathsf{he}},\mathsf{o}}\} \cup \{f_{\mathsf{H}_{\mathsf{em}},\mathsf{i2},i}\}_{i\in[B]}$ and the running time of $\mathcal{A}_{\mathsf{cma}}$ is about that of $\mathcal{A}_{\mathsf{cmna}}$.

Proof. We use the sequence of games shown in Fig. 12.

Game G_0: This is the original $\mathcal{F}$-EUF-fCMNA game and we have

$$\Pr[W_0] = \mathrm{Adv}^{\mathcal{F}\text{-EUF-fCMA}}_{\text{ML-DSA}}(\mathcal{A}_{\mathsf{cma}}).$$

Game G_1: We replace KeyGen with an alternative algorithm KeyGen' that chooses (ρ, ρ', K) at random instead of computing them by H_{es}.

Lemma 13. *We have*

$$|\Pr[W_0] - \Pr[W_1]| \le \frac{q_{\mathsf{H}_{\mathsf{es}}}}{2^{127}}.$$

Proof. By replacing $\mathsf{H}_{\mathsf{es}}(\xi,\cdot,\cdot)$ with a random function, KeyGen transforms into KeyGen', and we have this lemma from [BHH+19, Corollary 1].

Game G_2: The database $\mathcal{Q}_{rnd,\mu}$ stores (rnd, μ) in FHSign. If the pair (rnd, μ) is already present in $\mathcal{Q}_{rnd,\mu}$, FHSign returns $\perp$. This modification ensures that the inputs to H_μ are always unique in FHSign. Note that G_2 and G_3 are preliminary steps for randomizing ρ'' in G_4.

Lemma 14. *We have*

$$|\Pr[W_1] - \Pr[W_2]| \le \frac{\binom{513}{2t}(q_{\mathsf{H}_\mu}+1)^3}{2^{502-2t}}.$$

Proof. When the adversary makes a query such that $(\overline{M'}, rnd) \notin \mathcal{Q}_{M',rnd}$ in Line 28 and $(rnd, \mu) \in \mathcal{Q}_{rnd,\mu}$ in Line 31, a difference arises between G_1 and G_2. Defining this event as **bad**, it follows that $|\Pr[W_1] - \Pr[W_2]| \le \Pr[\mathbf{bad}]$. When **bad** occurs, we have a distinct input pair such that $\phi_1(\mathsf{H}_\mu(\overline{tr}_1, \overline{M'_1})) = \phi_2(\mathsf{H}_\mu(\overline{tr}_2, \overline{M'_2}))$ and $\overline{M'_1} \neq \overline{M'_2}$ hold for some $\phi_1, \phi_2 \in \Phi$. Therefore, $\Pr[\mathbf{bad}]$ is bounded by the advantage of a quantum adversary $\mathcal{B}_{\mathsf{crf}}$ against collision resistance with fault injection, which has quantum oracle access to H_μ and is tasked with finding $(\phi_1, \phi_2, (\overline{tr}_1, \overline{M'_1}), (\overline{tr}_2, \overline{M'_2}))$ such that $\phi_1(\mathsf{H}_\mu(\overline{tr}_1, \overline{M'_1})) = \phi_2(\mathsf{H}_\mu(\overline{tr}_2, \overline{M'_2}))$ and $(\overline{tr}_1, \overline{M'_1}) \neq (\overline{tr}_2, \overline{M'_2})$. Therefore, we have this lemma from Lemma 5.

Game G_3: We modify the oracle access to the hedged extractor H_{he} by replacing it with a punctured oracle $\mathsf{H}_{\mathsf{he}}\backslash\mathcal{S}$ for $\mathcal{S} \coloneqq \{\phi(K) : \phi \in \Phi\} \times \mathbb{B}^{32} \times \mathbb{B}^{64}$ and G_3 outputs $\perp$ if $\mathbf{find} = \top$ (see the definitions of $\mathsf{H}\backslash\mathcal{S}$ and **find** in Definition 5). Then, the adversary cannot gain any information about the generation of ρ'' through oracle access to H_μ.

GAMES G_0-G_8

$\mathcal{Q}_{M'} := \emptyset$
$\mathcal{Q}_{M',rnd} := \emptyset$
$\mathcal{Q}_{rnd,\mu} := \emptyset$ // G_2-G_7
$\mathcal{Q}_{\rho'',\kappa} := \emptyset$ // G_5-G_7
find $:= \bot$ // G_3-G_8
find$' := \bot$ // G_6-G_8
$(pk, sk) \leftarrow \mathsf{KeyGen}(1^\lambda)$ // G_0
$(pk, sk) \leftarrow \mathsf{KeyGen}'(1^\lambda)$ // G_1-G_8
$(\rho, K, tr, sk_{\mathsf{ID}}) := \mathsf{skDecode}(sk)$
$\mathcal{S} := \{\phi(K) : \phi \in \Phi\} \times \mathbb{B}^{32} \times \mathbb{B}^{64}$ // G_3-G_8
for $i \in [q_{\mathsf{S}}]$ **do** // G_6-G_8
  $\rho_i'' \leftarrow_{\$} \mathbb{B}^{64}$ // G_6-G_8
$\mathcal{S}' := \{\phi(\rho_i'') : \phi \in \Phi, i \in [q_{\mathsf{S}}]\} \times [0, 2^{16} - 1]$ // G_6-G_8
$ctr := 1$ // G_6-G_8
$\mathrm{O} := \{\mathrm{FHSIGN}, |\mathsf{H}_{\mathsf{ch}}\rangle, |\mathsf{H}_\mu\rangle\}$
$\mathrm{O} := \mathrm{O} \cup \{|\mathsf{H}_{\mathsf{he}}\rangle, |\mathsf{H}_{\mathsf{em}}\rangle\}$ // G_0-G_2
$\mathrm{O} := \mathrm{O} \cup \{|\mathsf{H}_{\mathsf{he}}\backslash\mathcal{S}\rangle, |\mathsf{H}_{\mathsf{em}}\rangle\}$ // G_3-G_5
$\mathrm{O} := \mathrm{O} \cup \{|\mathsf{H}_{\mathsf{he}}\backslash\mathcal{S}\rangle, |\mathsf{H}_{\mathsf{em}}\backslash\mathcal{S}'\rangle\}$ // G_6-G_8
$(M^*, \sigma^*) \leftarrow \mathcal{A}^{\mathrm{O}}_{\mathsf{cmna}}(pk)$
if $M^* \in \mathcal{Q}_{M'}$ **then return** $\bot$
if find $= \top$ **then return** $\bot$ // G_3-G_8
if find$' = \top$ **then return** $\bot$ // G_6-G_8
return $\mathsf{Verify}(pk, M^*, \sigma^*)$

$\mathrm{FHSIGN}(M', rnd, f^*, \phi)$

$f^* := \phi$
$f := \mathsf{id}$ **for** $f \in \mathcal{F}\backslash\{f^*\}$
$(\hat{\boldsymbol{A}}, K, tr, sk_{\mathsf{ID}}) := \mathsf{T}_{sk}(sk)$
$(\overline{tr}, \overline{M'}) = f_{\mathsf{H}_\mu,\mathsf{i}}(tr, M')$
if $(\overline{M'}, rnd) \in \mathcal{Q}_{M',rnd}$ **then return** $\bot$
$\mathcal{Q}_{M',rnd} := \mathcal{Q}_{M',rnd} \cup \{(\overline{M'}, rnd)\}$
$\mu = f_{\mathsf{H}_\mu,\mathsf{o}}(\mathsf{H}_\mu(\overline{tr}, \overline{M'}))$
if $(rnd, \mu) \in \mathcal{Q}_{rnd,\mu}$ **then return** $\bot$ // G_2-G_7
$\mathcal{Q}_{rnd,\mu} := \mathcal{Q}_{rnd,\mu} \cup \{(rnd, \mu)\}$ // G_2-G_7
$\rho'' := f_{\mathsf{H}_{\mathsf{he}},\mathsf{o}}(\mathsf{H}_{\mathsf{he}}(f_{\mathsf{H}_{\mathsf{he}},\mathsf{i1}}(K), rnd, \mu))$ // G_0-G_3
$\rho'' \leftarrow_{\$} \mathbb{B}^{64}$ // G_4-G_5
$\rho'' := \rho''_{ctr}$, $ctr := ctr + 1$ // G_6-G_7
$\kappa := 0$, $i := 1$
repeat
  $(\kappa_j)_{j\in[\ell]} := (\kappa, \ldots, \kappa + \ell - 1)$ // G_0-G_7
  $(\rho''_j)_{j\in[\ell]} := f_{\mathsf{H}_{\mathsf{em}},\mathsf{i2},i}(\rho'', \ldots, \rho'')$ // G_0-G_7
  if $\exists j, (\rho''_j, \kappa_j) \in \mathcal{Q}_{\rho'',\kappa}$ **then return** $\bot$ // G_5-G_7
  $\mathcal{Q}_{\rho'',\kappa} := \mathcal{Q}_{\rho'',\kappa} \cup \{(\rho''_j, \kappa_j)\}_{j\in[\ell]}$ // G_5-G_7
  $\boldsymbol{y} := f_{\mathsf{H}_{\mathsf{em}},\mathsf{o},i}((\mathsf{H}_{\mathsf{em}}(\rho''_j, \kappa_j))_{j\in[\ell]})$ // G_0-G_6
  $\boldsymbol{y} \leftarrow_{\$} (S_\gamma)^\ell$ // G_7-G_8
  $(\boldsymbol{w}_1, st) := f_{\mathsf{P}_1,\mathsf{o},i}(\mathsf{P}_1(f_{\mathsf{P}_1,\mathsf{i},i}(\hat{\boldsymbol{A}}; \boldsymbol{y})))$
  $(\overline{\mu}, \overline{\boldsymbol{w}}_1) := f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}(\mu, \boldsymbol{w}_1)$
  $\overline{\tilde{c}} := f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}(\mathsf{H}_{\mathsf{ch}}(\overline{\mu}, \overline{\boldsymbol{w}}_1))$
  $c := f_{\mathsf{Sb},\mathsf{o},i}(\mathsf{Sb}(f_{\mathsf{Sb},\mathsf{i},i}(\overline{\tilde{c}})))$
  $(\boldsymbol{z}, st') := f_{\mathsf{P}_2,\mathsf{o},i}(\mathsf{P}_2(f_{\mathsf{P}_2,\mathsf{i},i}(sk_{\mathsf{ID}}, c, st)))$
  $(\overline{\boldsymbol{z}}, \boldsymbol{h}) := f_{\mathsf{Ab},\mathsf{o},i}(\mathsf{Ab}(f_{\mathsf{Ab},\mathsf{i},i}(sk_{\mathsf{ID}}, \boldsymbol{z}, st')))$
  $\kappa := \kappa + \ell$, $i := i + 1$
until $(\overline{\boldsymbol{z}}, \boldsymbol{h}) \neq \bot$
$\sigma := \mathsf{sigEncode}(\overline{\tilde{c}}, \overline{\boldsymbol{z}} \bmod^{\pm} q, \boldsymbol{h})$
$\mathcal{Q}_{M'} := \mathcal{Q}_{M'} \cup \{\overline{M'}\}$
return σ

Fig. 12: Games for $\mathcal{F}$-EUF-fCMA $\Rightarrow$ $\mathcal{F}'$-EUF-fCMNA

Lemma 15. *We have*

$$|\Pr[W_2] - \Pr[W_3]| \leq \frac{\sqrt{\sum_{i\in[t+1]}\binom{256}{i-1}}(q_{\mathsf{H}_{\mathsf{he}}} + 1)}{2^{127}}.$$

Proof. We use the semi-classical O2H lemma (see Lemma 2) to bound $|\Pr[W_2] - \Pr[W_3]|$. We denote by $\mathsf{H}'_{\mathsf{he}}$ a random function satisfying $\mathsf{H}'_{\mathsf{he}}(K, rnd, \mu) = \mathsf{H}_{\mathsf{he}}(K, rnd, \mu)$ for $(K, rnd, \mu) \notin \mathcal{S}$ and $\mathsf{H}'_{\mathsf{he}}$ outputs fresh random values for $(K, rnd, \mu) \in \mathcal{S}$. From Lemma 2, we have

$$|\Pr[W_2] - \Pr[W_3]| \leq \sqrt{(q_{\mathsf{H}_{\mathsf{he}}} + 1)\Pr[\mathcal{A}^{\mathrm{O}'}_{\mathsf{cmna}}(pk) : \textbf{find} = \top]}, \tag{1}$$

where $\mathrm{O}' = \{\mathrm{FHSIGN}, |\mathsf{H}_{\mathsf{ch}}\rangle, |\mathsf{H}_\mu\rangle, |\mathsf{H}'_{\mathsf{he}}\backslash\mathcal{S}\rangle, |\mathsf{H}_{\mathsf{em}}\rangle\}$.

Then, we can establish the bound on the probability of **find** $= \top$ by applying Lemma 3. We consider an adversary $\mathcal{B}_{\mathsf{o2h}}$ that attempts to find an element of $\mathcal{S}$ using oracle access to $\mathsf{H}'_{\mathsf{he}}$. Selecting $j \leftarrow_{\$} [q_{\mathsf{H}_{\mathsf{he}}}]$, $\mathcal{B}_{\mathsf{o2h}}$ executes $\mathcal{A}^{\mathrm{O}}_{\mathsf{cmna}}(pk)$, where $\mathcal{A}_{\mathsf{cmna}}$ plays G_2 with access to the oracle $\mathsf{H}'_{\mathsf{he}}$ instead of H_{he}. Note that $\mathcal{B}_{\mathsf{o2h}}$ chooses random ρ'' instead of generating $\rho'' := \mathsf{H}_{\mathsf{he}}(\phi(K), rnd, \mu)$. Since each query input (rnd, μ) is unique due to $(rnd, \mu) \notin \mathcal{Q}_{rnd,\mu}$ and $\mathcal{A}_{\mathsf{cmna}}$ has no access to H_{he}, the output of H_{he} is indistinguishable from random values, making this modification feasible. Just before $\mathcal{A}_{\mathsf{cmna}}$ makes its j-th query to $\mathsf{H}'_{\mathsf{he}}$, $\mathcal{B}_{\mathsf{o2h}}$ measures the query input register of $\mathcal{A}_{\mathsf{cmna}}$ and outputs the measurement result as (K^*, rnd^*, μ^*). The probability that $\mathcal{B}_{\mathsf{o2h}}$ outputs $(K^*, rnd^*, \mu^*) \in \mathcal{S}$ is 2^{-256} times the number of possible values for $\phi(K)$ for fixed K, that is, $\sum_{i\in[t+1]}\binom{256}{i-1}$. From Lemma 3, we have

$$\Pr\left[\mathcal{A}^{\mathrm{O}'}_{\mathsf{cmna}}(pk) : \textbf{find} = \top\right] \leq 4q_{\mathsf{H}_{\mathsf{he}}}\frac{\sum_{i\in[t+1]}\binom{256}{i-1}}{2^{256}}. \tag{2}$$

Combining Eqs. (1) and (2), we have this lemma.

GAME G_4: We choose ρ'' randomly instead of generating it via H_{he}. Since the inputs to H_μ are always unique and $\mathcal{A}_{\mathsf{cnma}}$ cannot obtain the values of $\mathsf{H}_{\mathsf{he}}(\phi(K), rnd, \mu)$ for all $\phi \in \Phi$ if $\mathbf{find} = \bot$, this modification does not change the adversary's view. Therefore, $\Pr[W_3] = \Pr[W_4]$ holds.

GAME G_5: Let $\mathcal{Q}_{\rho'',\kappa}$ be a set keeping inputs $\{(\rho''_j, \kappa_j)\}_{j\in[\ell]}$ for H_{em} in FHSIGN. If an input (ρ''_j, κ_j) for H_{em} already exists in $\mathcal{Q}_{\rho'',\kappa}$, FHSIGN outputs $\bot$. This modification ensures that the inputs to H_{em} are always unique in FHSIGN. Note that G_5 and G_6 are preliminary steps for randomizing $\boldsymbol{y}$ in G_7.

Lemma 16. *We have*

$$|\Pr[W_4] - \Pr[W_5]| \leq \frac{\sum_{i\in[2t+1]}\binom{512}{i-1}{q_{\mathsf{S}}}^2}{2^{513}}.$$

Proof. Since we can bound $|\Pr[W_4] - \Pr[W_5]|$ by the collision probability of selecting ρ'' with up to $2t$-bit differences, the bound on the probability follows from the proof of Lemma 8.

GAME G_6: At the beginning of the game, the challenger selects ρ''_i for each $i \in [q_{\mathsf{S}}]$ and uses ρ''_i in the i-th query to FHSIGN. Additionally, we modify the oracle access to H_{em} by replacing it with an oracle of H_{em} punctured on a set $\mathcal{S}' := \{\phi(\rho''_i) : \phi \in \Phi, i \in [q_{\mathsf{S}}]\} \times [0, 2^{16} - 1]$ and G_6 outputs $\bot$ if $\mathbf{find}' = \top$, where $\mathbf{find}'$ corresponds to $\mathsf{H}_{\mathsf{em}}\backslash\mathcal{S}'$. Then, the adversary cannot gain any information about the generation of $\boldsymbol{y}$ through oracle access to H_{em} when it wins.

Lemma 17. *We have*

$$|\Pr[W_5] - \Pr[W_6]| \leq \frac{\sqrt{\sum_{i\in[t+1]}\binom{512}{i-1}{q_{\mathsf{S}}}(q_{\mathsf{H}_{\mathsf{em}}} + 1)}}{2^{255}}.$$

Proof. The proof almost follows Lemma 15. We use Lemma 2 to bound $|\Pr[W_5] - \Pr[W_6]|$. Let $\mathsf{H}'_{\mathsf{em}}$ be a random function satisfying $\mathsf{H}'_{\mathsf{em}}(\rho'', \kappa) = \mathsf{H}_{\mathsf{he}}(\rho'', \kappa)$ for $(\rho'', \kappa) \notin \mathcal{S}'$ and $\mathsf{H}'_{\mathsf{em}}(\rho'', \kappa)$ outputs random values for $(\rho'', \kappa) \in \mathcal{S}'$. From Lemma 2, we have

$$|\Pr[W_5] - \Pr[W_6]| \leq \sqrt{(q_{\mathsf{H}_{\mathsf{em}}} + 1)\Pr[\mathcal{A}^{\mathrm{O}''}_{\mathsf{cmna}}(pk) : \mathbf{find}' = \top]}, \tag{3}$$

where $\mathrm{O}'' = \{\text{FHSIGN}, |\mathsf{H}_{\mathsf{ch}}\rangle, |\mathsf{H}_\mu\rangle, |\mathsf{H}_{\mathsf{he}}\backslash\mathcal{S}\rangle, |\mathsf{H}'_{\mathsf{em}}\backslash\mathcal{S}'\rangle\}$.

We derive the bound on the probability of $\mathbf{find}' = \top$ by using Lemma 3. Let $\mathcal{B}'_{\mathsf{o2h}}$ be an adversary who tries to find an element of $\mathcal{S}'$ with oracle access to $\mathsf{H}'_{\mathsf{he}}$. Selecting $j \leftarrow_{\$} [q_{\mathsf{H}_{\mathsf{em}}}]$, $\mathcal{B}_{\mathsf{o2h}}$ executes $\mathcal{A}^{\mathrm{O}}_{\mathsf{cmna}}(pk)$, where $\mathcal{A}_{\mathsf{cmna}}$ plays G_5 with access to the oracle $\mathsf{H}'_{\mathsf{em}}$. Note that $\mathcal{B}'_{\mathsf{o2h}}$ chooses random $\boldsymbol{y}$. Since $(\rho'', \kappa) \notin \mathcal{Q}_{\rho'',\kappa}$ holds when $\boldsymbol{y}$ is chosen, each input for H_{em} is unique. Also, $\mathcal{A}_{\mathsf{cma}}$ has no access to H_{em}. Therefore, the output of H_{em} is indistinguishable from random values. Just before $\mathcal{A}_{\mathsf{cmna}}$ makes its j-th query to $\mathsf{H}'_{\mathsf{em}}$, $\mathcal{B}'_{\mathsf{o2h}}$ measures the query input register of $\mathcal{A}_{\mathsf{cmna}}$ and outputs the measurement result as (ρ^*, κ^*). The probability that $\mathcal{B}'_{\mathsf{o2h}}$ outputs $(\rho^*, \kappa^*) \in \mathcal{S}'$ is $\sum_{i\in[t+1]}\binom{512}{i-1}{q_{\mathsf{S}}} \cdot 2^{-512}$, as there are $\sum_{i\in[t+1]}\binom{512}{i-1}$ possible values for $\phi(\rho'')$ and at most q_{S} distinct values for ρ'' can be chosen. From Lemma 3, we have

$$\Pr\left[\mathcal{A}^{\mathrm{O}''}_{\mathsf{cmna}}(pk) : \mathbf{find}' = \top\right] \leq 4q_{\mathsf{H}_{\mathsf{he}}}\frac{\sum_{i\in[t+1]}\binom{512}{i-1}{q_{\mathsf{S}}}}{2^{512}}. \tag{4}$$

Combining Eqs. (3) and (4), we have this lemma.

GAME G_7: We choose $\boldsymbol{y}$ randomly instead of generating it using H_{em}. Since inputs to H_{em} are always unique and $\mathcal{A}_{\mathsf{cnma}}$ cannot obtain the values of $(\mathsf{H}_{\mathsf{em}}(\rho''_j, \kappa_j))_{j\in[\ell]}$ if $\mathbf{find}' = \bot$ holds, this modification does not change the adversary's view. Therefore, $\Pr[W_6] = \Pr[W_7]$ holds.

GAME G_8: We finally remove $\mathcal{Q}_{rnd,\mu}$ and $\mathcal{Q}_{\rho'',\kappa}$ from the game; as a result, FHSIGN no longer verifies $(rnd, \mu) \in \mathcal{Q}_{rnd,\mu}$ and $(\rho''_i, \kappa_i) \in \mathcal{Q}_{\rho'',\kappa}$. A difference arises only if FHSIGN returns $\bot$ in Lines 31 and 40 of G_7. Thus, we can bound:

$$|\Pr[W_7] - \Pr[W_8]| \leq \frac{\binom{513}{2t}(q_{\mathsf{H}_\mu} + 1)^3}{2^{502-2t}} + \frac{\sum_{i\in[2t+1]}\binom{512}{i-1}{q_{\mathsf{S}}}^2}{2^{513}},$$

using Lemmas 14 and 16.

Due to this last modification, FHSIGN of G_8 becomes identical to the one of the EUF-fCMA game.

Now, $\mathcal{A}_{\mathsf{cma}}$ can simulate G_8 since FHSign in G_8 is identical to FSign in the $\mathcal{F}$-EUF-fCMA game, and puncturing on H_{he} and H_{em} is feasible.

Lemma 18. *There exists an* $\mathcal{F}$-EUF-fCMA *adversary* $\mathcal{A}_{\mathsf{cma}}$ *of* sML-DSA *such that*

$$\Pr[W_8] \leq \mathrm{Adv}^{\text{EUF-fCMA}}_{\text{sML-DSA}}(\mathcal{A}_{\mathsf{cma}}).$$

Proof. The $\mathcal{F}$-EUF-fCMA adversary $\mathcal{A}_{\mathsf{cma}}$ directly utilizes its oracle FSign to answer queries from $\mathcal{A}_{\mathsf{cmna}}$ playing G_8. In this simulation, $\mathcal{A}_{\mathsf{cma}}$ generates $pk = \mathsf{pkEncode}(\rho, \boldsymbol{t}_1)$ from $pk' = \mathsf{pkEncode}'(\rho, \boldsymbol{t}_0, \boldsymbol{t}_1)$ and computes tr as in Lemma 12. Additionally, $\mathcal{A}_{\mathsf{cma}}$ selects a random K and $\{\rho''_i\}_{i\in[q_{\mathsf{S}}]}$, which are used to define $\mathcal{S}$ and $\mathcal{S}'$. Since K and $\{\rho''_i\}_{i\in[q_{\mathsf{S}}]}$ are not used in FHSign, the oracle usage by $\mathcal{A}_{\mathsf{cma}}$ remains valid. Thus, the lemma follows.

□

Remark 2. Theorems 1 and 2 apply to the EUF-CMNA (Chosen Message and Nonce Attack) model, which can be defined by disabling fault injection in Definition 6.

Remark 3. The dominant terms in the security bounds of Theorems 1 and 2 are $\sqrt{\sum_{i\in[t+1]}\binom{256}{i-1}}(q_{\mathsf{H}_{\mathsf{he}}}+1)\cdot 2^{-127}$ and $q_{\mathsf{H}_{\mathsf{es}}}\cdot 2^{-127}$. As the number of tampered bits increases, this first term can no longer be regarded as negligible. A detailed analysis of this effect is provided in Appendix E We suggest the following countermeasures to mitigate the issue.

- Increase the length of K from 256 bits to 512 bits, which reduces the dominant term to $\sqrt{\sum_{i\in[t+1]}\binom{512}{i-1}}(q_{\mathsf{H}_{\mathsf{he}}}+1)\cdot 2^{-255}$. Although this modification deviates from the FIPS 204 specification, it provides a practical and interoperable countermeasure.
- Implement physical countermeasures against multi-bit tampering.

Note that we can assume the pseudorandomness of H_{he} in the standard model, in which case related-key attacks on H_{he} must be taken into account [BK03].

6 Key-Recovery Attacks Exploiting Faults without Proven Resilience

In Section 5, we demonstrated that ML-DSA is $\mathcal{F}'$-EUF-fCMNA-secure in the QROM. Here, we discuss attacks on ML-DSA involving faults for which resilience cannot be proven. While this section provides a theoretical evaluation, a survey of practical attacks is presented in Appendix A.

We present attacks that recover $\boldsymbol{s}_1$, from which the entire secret $(\boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{t}_0)$ can be computed. For simplicity, we assume in the following discussion that we can correctly guess the number of loop iterations required to generate a signature.

$f_{\mathsf{H}_{\mathsf{he}},\mathsf{i2}}$: Tampering the input (rnd, μ) to H_{he}, via $f_{\mathsf{H}_{\mathsf{he}},\mathsf{i2}}$ allows us to obtain two signatures with the same randomness, which leads to the special soundness attack. Let us fix (M', rnd). Injecting a fault in $\tilde{c}$ via $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}$, $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}$, or $f_{\mathsf{P}_2,\mathsf{i},i}$, we obtain a response $\boldsymbol{z}_1$ corresponding to faulty $\bar{\tilde{c}}$ and $\boldsymbol{y}$ derived from (M', rnd). We then choose a pair $(\phi, \overline{rnd})$ such that $\phi(\overline{rnd}) = rnd$. Querying $(M', \overline{rnd})$ with $f_{\mathsf{H}_{\mathsf{he}},\mathsf{i2}} = \phi$, we obtain $\boldsymbol{z}_2$ corresponding to $\tilde{c}$ and $\boldsymbol{y}$. Thus, we can mount the special soundness attack to obtain $\boldsymbol{s}_1$.

$f_{\mathsf{H}_{\mathsf{em}},\mathsf{i1},i}$: Controlling the counter for the expand mask, we can mount the attack proposed by ElGhamrawy et al. [EAB+23]. Let $\boldsymbol{v}[i]$ denote the i-th element of the vector $\boldsymbol{v}$. By tampering with the 1-bit of κ_2 $(= \kappa_1 + 1)$, we can set $\kappa_1 = \kappa_2$. In this case, the outputs of H_{em} will satisfy $\boldsymbol{y}[1] = \boldsymbol{y}[2]$, meaning the first polynomial is repeated. Using this equality, we obtain the relation $\boldsymbol{z}[1] - \boldsymbol{z}[2] = c\cdot(\boldsymbol{s}_1[1] - \boldsymbol{s}_1[2])$.[16] Those relations help us to obtain $\boldsymbol{s}_1$.

$f_{\mathsf{H}_{\mathsf{em}},\mathsf{o},i}$, $f_{\mathsf{P}_1,\mathsf{i},i}$, and $f_{\mathsf{P}_1,\mathsf{o},i}$: Liu et al. [LZS+21] and Damm et al. [DKM+25] gave a key-recovery attack by using *side-channel analysis (SCA)* against Dilithium [LDK+17] and qTESLA [BAA+17], which is also applicable to ML-DSA. Their SCA-based attack exploits the 1-bit leakage of $\boldsymbol{y}$ and the knowledge of $\boldsymbol{z}$, and corrects several signatures with the leakages to build an instance of the Integer LWE

[16] We can notice the success of guessing the number of loops since $\|c\cdot(\boldsymbol{s}_1[1] - \boldsymbol{s}_1[2])\|_\infty \leq 2\tau\eta$ is much smaller than the other differences in the concrete parameters.

problem [BDE+18]. Thus, by fixing a bit of $\boldsymbol{y}$ via $f_{\mathsf{H_{em}},\mathsf{o},i}$, $f_{\mathsf{P}_1,\mathsf{i},i}$, or $f_{\mathsf{P}_1,\mathsf{o},i}$, we can mimic the 1-bit leakage and mount their attack in the EUF-fCMNA setting if we can correctly guess the number of loops. Note that, in the real environment, we can use timing information to check the correctness of our guessing.

Faults on $\hat{\boldsymbol{A}}$ via $f_{\mathsf{P}_1,\mathsf{i},i}$ (or $f_{\mathsf{T}_{sk},\mathsf{o}}$) can also be exploited to recover a coefficient of $\boldsymbol{y}$, as demonstrated by Krahmer et al. [KPLG24].

$f_{\mathsf{Sb},\mathsf{o},i}$: We discuss that controlling the challenge c enables us to leak the information of the secret key $\boldsymbol{s}_1$: We can inject faults into c and expect that the absolute values of the coefficients of $c\boldsymbol{s}_1$ are abnormally larger than the worst-case threshold $\beta := \tau\eta$.[17] If a coefficient of $\boldsymbol{z}$ is $\gamma_1 - \Delta$ (or $-\gamma_1 + \Delta + 1$, resp.) for some $\Delta > \beta$, then we can decide that the corresponding coefficient of $c\boldsymbol{s}_1$ is at least $-\Delta$ (or at most Δ, resp.) since the coefficients of $\boldsymbol{y}$ are at most γ_1 (or at least $-\gamma_1 + 1$, resp.). That information allows us to narrow the space of $\boldsymbol{s}_1$. Estimating the concrete query complexity and computational cost of such attacks is left as an open problem for future work.

$f_{\mathsf{T}_{sk},\mathsf{i}}$, $f_{\mathsf{T}_{sk},\mathsf{o}}$, $f_{\mathsf{P}_2,\mathsf{i},i}$: It is trivial to recover the bits of $\boldsymbol{s}_1$ by checking whether the signature, generated by fixing the bits via $f_{\mathsf{T}_{sk},\mathsf{i}}$ or $f_{\mathsf{T}_{sk},\mathsf{o}}$, passes verification. We can extend it to in-loop faults via $f_{\mathsf{P}_2,\mathsf{i},i}$. We select a position of a bit setting 0, and verify the generated signature. If all B verification passes, then it can be concluded that the specific bit in $\boldsymbol{s}_1$ is 0. Otherwise, there should be one $i \in [B]$ such that the verification fails, and the selected bit should be 1. Repeating this process for the number of bits in $\boldsymbol{s}_1$, we can recover $\boldsymbol{s}_1$.

$f_{\mathsf{P}_2,\mathsf{o},i}$, $f_{\mathsf{Ab},\mathsf{i},i}$: By injecting a fault into the response $\boldsymbol{z}$ passed to the abort procedure Ab, we can produce a faulty $\overline{\boldsymbol{z}}$ that would normally satisfy the condition check and obtain a rejected signature including $\boldsymbol{z}$ by reversing the fault. Such a response $\boldsymbol{z}$ may leak information about the secret value $\boldsymbol{s}_1$ or $\boldsymbol{s}_2$. Indeed, Azevedo-Oliveira et al. [AOCVCG25] proposed key-recovery attacks on skipping the first or second condition checks (Line 28 in Fig. 5). Concretely speaking, we flip a bit of $\boldsymbol{z}$ via $f_{\mathsf{P}_2,\mathsf{o},i}$ or $f_{\mathsf{Ab},\mathsf{i},i}$ to make the maximum norm of $\overline{\boldsymbol{z}}$ small. After obtaining a tampered $\overline{\boldsymbol{z}}$, we flip the same bit of $\overline{\boldsymbol{z}}$ to obtain the original $\boldsymbol{z}$. If the norm of $\boldsymbol{z}$ exceeds γ_1, it becomes usable for recovering $\boldsymbol{s}_1$.

In addition, faults on $\boldsymbol{w}$ can bypass the second condition check for the size of $\boldsymbol{r}_0$. As a result, the key-recovery attack based on skipping the second condition check, proposed by Azevedo-Oliveira et al. [AOCVCG25], can be mounted.

Conversely, we provide an intuitive explanation in Appendix D for why ML-DSA remains secure against faults other than those discussed above.

Acknowledgment

The authors would like to thank to anonymous reviewers of Asiacrypt 2025 for their helpful and insightful comments. The authors used Grammarly for grammar/spelling checking.

References

ABF+18. Christopher Ambrose, Joppe W. Bos, Björn Fay, Marc Joye, Manfred Lochter, and Bruce Murray. Differential attacks on deterministic signatures. In Nigel P. Smart, editor, *CT-RSA 2018*, volume 10808 of *LNCS*, pages 339–353. Springer, Cham, April 2018. 2

AFLT16. Michel Abdalla, Pierre-Alain Fouque, Vadim Lyubashevsky, and Mehdi Tibouchi. Tightly secure signatures from lossy identification schemes. *Journal of Cryptology*, 29(3):597–631, July 2016. 2, 7

AHU19. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Boldyreva and Micciancio [BM19], pages 269–295. 5, 8, 9, 27, 28, 29

AOCVCG25. Paco Azevedo-Oliveira, Andersson Calle Viera, Benoît Cogliati, and Louis Goubin. Finding a polytope: A practical fault attack against dilithium. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025*, volume 15674 of *LNCS*, pages 259–283, Cham, May 2025. Springer. 6, 22

AOTZ20. Diego F. Aranha, Claudio Orlandi, Akira Takahashi, and Greg Zaverucha. Security of hedged Fiat-Shamir signatures under fault attacks. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 644–674. Springer, Cham, May 2020. 2, 3, 4, 5, 7, 11, 12, 25, 26, 27

[17] It depends on the implementation. In the reference/avx2 implementation of Dilithium, c's coefficients are of type int32_t, so even a 1-bit fault can be effective.

AWK+25. Samy Amer, Yingchen Wang, Hunter Kippen, Thinh Dang, Daniel Genkin, Andrew Kwong, Alexander Nelson, and Arkady Yerukhimovich. PQ-hammer: End-to-end key recovery attacks on post-quantum cryptography using rowhammer. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 48–48, Los Alamitos, CA, USA, May 2025. IEEE Computer Society. 2, 25, 26

BAA+17. Nina Bindel, Sedat Akleylek, Erdem Alkim, Paulo S. L. M. Barreto, Johannes Buchmann, Edward Eaton, Gus Gutoski, Juliane Kramer, Patrick Longa, Harun Polat, Jefferson E. Ricardini, and Gustavo Zanon. qTESLA. Technical report, National Institute of Standards and Technology, 2017. available at https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-1-submissions. 21

BBD+23. Manuel Barbosa, Gilles Barthe, Christian Doczkal, Jelle Don, Serge Fehr, Benjamin Grégoire, Yu-Hsuan Huang, Andreas Hülsing, Yi Lee, and Xiaodi Wu. Fixing and mechanizing the security proof of Fiat-Shamir with aborts and Dilithium. In Handschuh and Lysyanskaya [HL23], pages 358–389. 2, 5, 11, 30, 31

BDE+18. Jonathan Bootle, Claire Delaplace, Thomas Espitau, Pierre-Alain Fouque, and Mehdi Tibouchi. LWE without modular reduction and improved side-channel attacks against BLISS. In Thomas Peyrin and Steven Galbraith, editors, *ASIACRYPT 2018, Part I*, volume 11272 of *LNCS*, pages 494–524. Springer, Cham, December 2018. 22

BDL01. Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the importance of eliminating errors in cryptographic computations. *Journal of Cryptology*, 14(2):101–119, March 2001. 6

BDL+11. Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. In Bart Preneel and Tsuyoshi Takagi, editors, *CHES 2011*, volume 6917 of *LNCS*, pages 124–142. Springer, Berlin, Heidelberg, September / October 2011. 2

BG14. Shi Bai and Steven D. Galbraith. An improved compression technique for signatures based on learning with errors. In Josh Benaloh, editor, *CT-RSA 2014*, volume 8366 of *LNCS*, pages 28–47. Springer, Cham, February 2014. 9

BHH+19. Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 61–90. Springer, Cham, December 2019. 8, 18

BK03. Mihir Bellare and Tadayoshi Kohno. A theoretical treatment of related-key attacks: RKA-PRPs, RKA-PRFs, and applications. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 491–506. Springer, Berlin, Heidelberg, May 2003. 21

BM19. Alexandra Boldyreva and Daniele Micciancio, editors. *CRYPTO 2019, Part II*, volume 11693 of *LNCS*. Springer, Cham, August 2019. 22, 23

BP18. Leon Groot Bruinderink and Peter Pessl. Differential fault attacks on deterministic lattice signatures. *IACR TCHES*, 2018(3):21–43, 2018. 2, 25, 26

BT16. Mihir Bellare and Björn Tackmann. Nonce-based cryptography: Retaining security when randomness fails. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part I*, volume 9665 of *LNCS*, pages 729–757. Springer, Berlin, Heidelberg, May 2016. 2, 27

CDF+21. Cas Cremers, Samed Düzlü, Rune Fiedler, Marc Fischlin, and Christian Janson. BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures. In *2021 IEEE Symposium on Security and Privacy*, pages 1696–1714. IEEE Computer Society Press, May 2021. 9

CGV+21. Brice Colombier, Paul Grandamme, Julien Vernay, Émilie Chanavat, Lilian Bossuet, Lucie de Laulanié, and Bruno Chassagne. Multi-spot laser fault injection setup: New possibilities for fault injection attacks. In Vincent Grosso and Thomas Pöppelmann, editors, *CARDIS 2021*, volume 13173 of *LNCS*, pages 151–166. Springer, 2021. 3

DFMS19. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Boldyreva and Micciancio [BM19], pages 356–383. 3

DFPS23. Julien Devevey, Pouria Fallahpour, Alain Passelègue, and Damien Stehlé. A detailed analysis of Fiat-Shamir with aborts. In Handschuh and Lysyanskaya [HL23], pages 327–357. 2, 5, 7

DKM+25. Simon Damm, Nicolai Kraus, Alexander May, Julian Nowakowski, and Jonas Thietke. One bit to rule them all – imperfect randomness harms lattice signatures. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025*, volume 15674 of *LNCS*, pages 284–316, Cham, May 2025. Springer. 21

EAB+23. Mohamed ElGhamrawy, Melissa Azouaoui, Olivier Bronchain, Joost Renes, Tobias Schneider, Markus Schönauer, Okan Seker, and Christine van Vredendaal. From MLWE to RLWE: A differential fault attack on randomized & deterministic Dilithium. *IACR TCHES*, 2023(4):262–286, 2023. 21, 26, 27

EFGT16. Thomas Espitau, Pierre-Alain Fouque, Benoît Gérard, and Mehdi Tibouchi. Loop-abort faults on lattice-based Fiat-Shamir and hash-and-sign signatures. In Roberto Avanzi and Howard M. Heys, editors, *SAC 2016*, volume 10532 of *LNCS*, pages 140–158. Springer, Cham, August 2016. 27

FG20. Marc Fischlin and Felix Günther. Modeling memory faults in signature and authenticated encryption schemes. In Stanislaw Jarecki, editor, *CT-RSA 2020*, volume 12006 of *LNCS*, pages 56–84. Springer, Cham, February 2020. 2

FS87. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987. 1

GHHM21. Alex B. Grilo, Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Tight adaptive reprogramming in the QROM. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part I*, volume 13090 of *LNCS*, pages 637–667. Springer, Cham, December 2021. 2, 3, 5, 8

GMR88. Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.*, 17(2):281–308, 1988. 1

HL23. Helena Handschuh and Anna Lysyanskaya, editors. *CRYPTO 2023, Part V*, volume 14085 of *LNCS*. Springer, Cham, August 2023. 23

IMS+22. Saad Islam, Koksal Mus, Richa Singh, Patrick Schaumont, and Berk Sunar. Signature correction attack on dilithium signature scheme. In *2022 IEEE European Symposium on Security and Privacy*, pages 647–663. IEEE Computer Society Press, June 2022. 26, 27

JMD24. Sönke Jendral, John Preuß Mattsson, and Elena Dubrova. A single-trace fault injection attack on hedged module lattice digital signature algorithm (ML-DSA). In *FDTC 2024*, pages 34–43, 2024. 26

JMW24. Kelsey A. Jackson, Carl A. Miller, and Daochen Wang. Evaluating the security of CRYSTALS-dilithium in the quantum random oracle model. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 418–446. Springer, Cham, May 2024. 3

KLS18. Eike Kiltz, Vadim Lyubashevsky, and Christian Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 552–586. Springer, Cham, April / May 2018. 2, 3, 5, 7, 27, 29, 30

KPLG24. Elisabeth Krahmer, Peter Pessl, Georg Land, and Tim Güneysu. Correction fault attacks on randomized CRYSTALS-dilithium. *IACR TCHES*, 2024(3):174–199, 2024. 22, 26

KX24a. Haruhisa Kosuge and Keita Xagawa. Probabilistic hash-and-sign with retry in the quantum random oracle model. In Qiang Tang and Vanessa Teague, editors, *PKC 2024, Part I*, volume 14601 of *LNCS*, pages 259–288. Springer, Cham, April 2024. 5, 14

KX24b. Mukul Kulkarni and Keita Xagawa. Strong existential unforgeability and more of MPC-in-the-head signatures. Cryptology ePrint Archive, Report 2024/1069, 2024. 29

LDK+17. Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2017. available at https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-1-submissions. 21

Lyu09. Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, Berlin, Heidelberg, December 2009. 1

Lyu12. Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, Berlin, Heidelberg, April 2012. 1

LZS+21. Yuejun Liu, Yongbin Zhou, Shuo Sun, Tianyu Wang, Rui Zhang, and Jingdian Ming. On the security of lattice-based Fiat-Shamir signatures in the presence of randomness leakage. *IEEE Transactions on Information Forensics and Security*, 16:1868–1879, 2021. 21

Nat24a. National Institute of Standards and Technology. FIPS 204, module-lattice-based digital signature standard. Standard, U.S. Department of Commerce, Washington, D.C., August 2024. 1, 2, 6, 9, 10, 11

Nat24b. National Institute of Standards and Technology. FIPS 205, stateless hash-based digital signature standard. Standard, U.S. Department of Commerce, Washington, D.C., August 2024. 1

Per16. Trevor Perrin. The XEdDSA and VXEdDSA signature schemes. Specification, Signal, October 2016. Rev.1. 2

Por13. Thomas Pornin. Deterministic usage of the digital signature algorithm (DSA) and elliptic curve digital signature algorithm (ECDSA). RFC 6979, August 2013. 2

PSS+18. Damian Poddebniak, Juraj Somorovsky, Sebastian Schinzel, Manfred Lochter, and Paul Rösler. Attacking deterministic signature schemes using fault attacks. In *2018 IEEE European Symposium on Security and Privacy*, pages 338–352. IEEE Computer Society Press, April 2018. 2

RJH+19. Prasanna Ravi, Mahabir Prasad Jhanwar, James Howe, Anupam Chattopadhyay, and Shivam Bhasin. Exploiting determinism in lattice-based signatures: Practical fault attacks on pqm4 implementations of NIST candidates. In Steven D. Galbraith, Giovanni Russello, Willy Susilo, Dieter

Gollmann, Engin Kirda, and Zhenkai Liang, editors, *ASIACCS 19*, pages 427–440. ACM Press, July 2019. 26

RP17. Yolan Romailler and Sylvain Pelissier. Practical fault attack against the Ed25519 and EdDSA signature schemes. In *FDTC 2017*, pages 17–24. IEEE Computer Society, 2017. 2

SB18. Niels Samwel and Lejla Batina. Practical fault injection on deterministic signatures: The case of EdDSA. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *AFRICACRYPT 18*, volume 10831 of *LNCS*, pages 306–321. Springer, Cham, May 2018. 2

Sho94. Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th FOCS*, pages 124–134. IEEE Computer Society Press, November 1994. 1

SHS16. Bodo Selmke, Johann Heyszl, and Georg Sigl. Attack on a DFA protected AES by simultaneous laser fault injections. In *FDTC 2016*, pages 36–46. IEEE Computer Society, 2016. 3

UMB+23. Vincent Quentin Ulitzsch, Soundes Marzougui, Alexis Bagia, Mehdi Tibouchi, and Jean-Pierre Seifert. Loop aborts strike back: Defeating fault countermeasures in lattice signatures with ILP. *IACR TCHES*, 2023(4):367–392, 2023. 26, 27

Xag24. Keita Xagawa. Signatures with memory-tight security in the quantum random oracle model. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VII*, volume 14657 of *LNCS*, pages 30–58. Springer, Cham, May 2024. 5

Zha15. Mark Zhandry. A note on the quantum collision and set equality problems. *Quantum Information and Computation*, 15(7&8):557–567, 2015. 5, 29

A Fault-injection Attacks on ML-DSA/Dilithium

We present a survey of fault-injection attacks on ML-DSA/Dilithium and give the summary in Table 2. For fault-injection attacks against Fiat-Shamir signatures, we adopt the classification introduced by Aranha et al. [AOTZ20, Sec. 3.1], which includes the special soundness attack (SSND), the large randomness bias attack (LRB), the randomness reuse attack (RR), and the invalid response attack (IR). Additionally, we introduce a lowering entropy attack (LE), which significantly reduces the randomness's entropy. We briefly summarize their discussion and provide additional insights on recent lowering entropy attacks. For simplicity, we omit the hint information $\boldsymbol{h}$ in the following. As in Section 6, the following attacks recover $\boldsymbol{s}_1$, which leads to the knowledge of the entire secret $(\boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{t}_0)$.

Special Soundness Attack (SSND): This attack exploits the special soundness of the ID scheme, which means that there is an efficient extractor outputting the witness sk from two valid proofs (w, c_1, z_1) and (w, c_2, z_2) with $c_1 \neq c_2$.

Let us consider the deterministic ML-DSA. Suppose that we obtain a siganture $\sigma_1 = (\tilde{c}_1, \boldsymbol{z}_1)$ for an encoded message M'. We then run the signature generation on the same M' with *fault* to use $\tilde{c}_2 \neq \tilde{c}_1$ as a challenge and obtain another signature $\sigma_2 = (\tilde{c}_2, \boldsymbol{z}_2)$. Since the deterministic ML-DSA uses the same $\boldsymbol{y}$ for both signatures, we have two equations: $\boldsymbol{z}_1 = \boldsymbol{y} + c_1\boldsymbol{s}_1$ and $\boldsymbol{z}_2 = \boldsymbol{y} + c_2\boldsymbol{s}_1$. Those two signatures allows us to compute $\boldsymbol{s}_1$ as $(\boldsymbol{z}_1 - \boldsymbol{z}_2) \cdot (c_1 - c_2)^{-1}$ if $c_1 - c_2$ is invertible.

There are several attack surfaces on which to mount SSND. Bruinderink and Pessl [BP18] analyzed fault-injection attacks on deterministic Dilithium to make $\tilde{c}_2$ different from $\tilde{c}_1$ as depicted in Table 2. Amer et al. [AWK+25] also gave a rowhammer-based attack against the deterministic ML-DSA, which flips some bits of ρ, the seed for $\hat{\boldsymbol{A}}$, to differ $\tilde{c}_2$ from $\tilde{c}_1$.

Large Randomness Bias Attack (LRB): This attack can be considered a variant of SSND and introduces an error to the randomness $\boldsymbol{y}$. Again, let us consider the deterministic ML-DSA. First, we obtain a signature $\sigma_1 = (\tilde{c}_1, \boldsymbol{z}_1)$ on an encoded message M'. Next, we modify $\boldsymbol{y}$ to $\boldsymbol{y} + \Delta$ with predictable Δ during the signature generation for the same message M', and obtain a second signature $\sigma_2 = (\tilde{c}_2, \boldsymbol{z}_2)$ using the modified randomness $\boldsymbol{y} + \Delta$. We then have $\boldsymbol{z}_1 = \boldsymbol{y} + c_1\boldsymbol{s}_1$ and $\boldsymbol{z}_2 = \boldsymbol{y} + \Delta + c_2\boldsymbol{s}_1$, where $c_1 \neq c_2$ with high probability because the commitment with faulty randomness $\boldsymbol{y} + \Delta$ differs from the original commitment with $\boldsymbol{y}$. Those two signatures allows us to compute $\boldsymbol{s}_1$ as $(\boldsymbol{z}_1 - \boldsymbol{z}_2 - \Delta) \cdot (c_1 - c_2)^{-1}$ if $c_1 - c_2$ is invertible.

Bruinderink and Pessl [BP18] provided this type of key-recovery attack against the deterministic Dilithium.

Table 2: Survey summary of fault-injection attacks on ML-DSA/Dilithium. "Fault Type" indicates the corresponding fault types in our model (defined in Section 4). "Attack Type" groups attacks following the definitions given in Appendix A. "Attack Vector" specifies the fault injection techniques. CBF/UBF (Controlled/Uncontrolled Bit-Flip) introduce faults into a value, with CBF targeting a predetermined range and UBF affecting any position. UIS/CIS (Uncontrolled/Controlled Instruction Skipping) are analogous techniques for corrupting specific computations during the signature generation process. In "Target", deterministic, random, and hedged are abbreviated as Det., Ran., and Heg., respectively.

Paper	Fault Type	Attack Type	Attack Vector	Target	Description
[BP18]-1	$f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}/f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}$	SSND	UBF/UIS	Det. Dilithium	Faulty hash input or output.
[BP18]-2	$f_{\mathsf{P}_1,\mathsf{o},i}$	SSND	UBF	Det. Dilithium	Faulty commitment computation to get a faulty hash input $\boldsymbol{w}_1$.
[BP18]-3	$f_{\mathsf{P}_1,\mathsf{i},i}$	SSND	UBF	Det. Dilithium	Faulty $\hat{\boldsymbol{A}}$ computed from sk as commitment input.
[AWK+25]	$f_{\mathsf{P}_1,\mathsf{i},i}$	SSND	UBF	Det. Dilithium	Faulty ρ'' as commitment input.
[BP18]-4	$f_{\mathsf{H}_{\mathsf{em}},\mathsf{o},i}/f_{\mathsf{P}_1,\mathsf{i},i}$	LRB	CBF	Det. Dilithium	Faulty $\boldsymbol{y}+\boldsymbol{\Delta}$ with small additive error.
[RJH+19]-1	-	IR	CIS	Det. Dilithium	Instruction skipping during the response computation to let $\boldsymbol{z}[i][j]=\boldsymbol{y}[i][j]$.
[RJH+19]-2	-	IR	CIS	Ran. Dilithium	Instruction skipping during the response computation to let $\boldsymbol{z}[i][j]=\langle\langle c\boldsymbol{s}_1\rangle\rangle[i][j]$.
[IMS+22]	$f_{\mathsf{P}_2,\mathsf{i},i}$	IR	UBF	Ran. Dilithium	Faulty $\boldsymbol{s}_1+\boldsymbol{\Delta}$ with small additive error as response input.
[KPLG24]-1	-	IR	CIS	Ran. Dilithium	Instruction skipping during the response computation to let $\boldsymbol{z}[i][j]=\boldsymbol{y}[i][j]$.
[KPLG24]-2	$f_{\mathsf{P}_1,\mathsf{i},i}$	IR	CIS	Ran. Dilithium	Faulty $\hat{\boldsymbol{A}}+\boldsymbol{\Delta}$ with small additive error.
[JMD24]	-	RR	CIS	Hed. ML-DSA	Instruction skipping during the computation of ρ'' to fix ρ''.
[UMB+23]	-/$f_{\mathsf{P}_1,\mathsf{i},i}$	LE	CIS	Ran. Dilithium	Instruction skipping during the randomness sampling to make $\overline{\boldsymbol{y}}$ sparse.
[EAB+23]	-/$f_{\mathsf{H}_{\mathsf{em}},\mathsf{i}1,i}/f_{\mathsf{P}_1,\mathsf{i},i}$	LE	CIS	Ran. Dilithium	Instruction skipping during randomness sampling may cause the randomness to become $(y,\ldots,y)$.

Randomness Reuse Attack (RR): Aranha et al. [AOTZ20, Sec.3.1] defines a randomness reuse attack (RR) as an attack fixing the randomness $\boldsymbol{y}$ to be a constant. This attack is very strong since it applies to the hedged ML-DSA. For example, Jendral et al. [JMD24] gave an instruction skipping fault during the computation of the private random seed ρ'' to fix ρ'' as a known constant. This allows us to compute $\boldsymbol{y} \coloneqq \mathsf{ExpandMask}(\rho'', \kappa)$ and obtain $\boldsymbol{s}_1 \coloneqq c^{-1} \cdot (\boldsymbol{z} - \boldsymbol{y})$ if c is invertible.

Invalid Response Attack (IR): The attack injects a fault during the computation of the response, which leads to an invalid response $\overline{\boldsymbol{z}}$. For example, Ravi et al. [RJH+19] proposed two variations of this attack depending on how the computation of $\boldsymbol{z} \coloneqq \boldsymbol{y} + \langle\langle c\boldsymbol{s}_1\rangle\rangle$ (Line 25 of Fig. 5) is implemented.

The first attack leads to $\overline{\boldsymbol{z}}[i][j] = \langle\langle c\boldsymbol{s}_1\rangle\rangle[i][j] = \langle \boldsymbol{s}_1[i], cx^{j-1}\rangle$ for some (i,j), where $\boldsymbol{v}[i][j]$ denotes the j-th coefficient of the i-th element of $\boldsymbol{v} \in R_q^\ell$ and $\langle v, w\rangle$ is the inner product of the vector of coefficients of two polynomials $v, w \in R_q$. Since c is publicly known, we can derive $\boldsymbol{s}_1[i]$ by solving the system of linear equations. We note that this attack applies to the hedged ML-DSA, while the authors only considered the deterministic Dilithium.

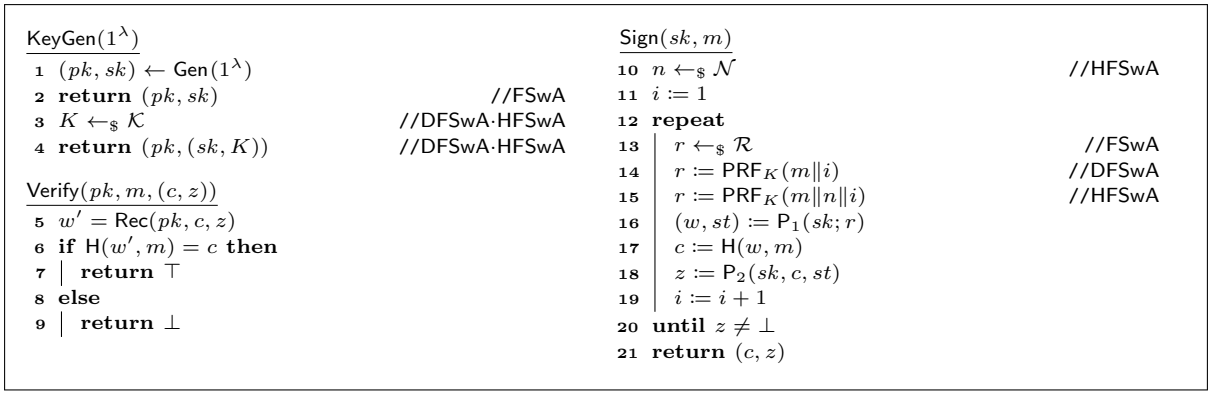

Fig. 13: Algorithms of Fiat-Shamir with aborts and its variants

The second attack leads to $\overline{\boldsymbol{z}}[i][j] = \boldsymbol{y}[i][j]$ for some i and j. In the deterministic ML-DSA/Dilithium, when signing the same message, the difference between the fault-free response $\boldsymbol{z}$ and the faulty response $\overline{\boldsymbol{z}}$ can reveal secret information as $\boldsymbol{z}[i][j] - \overline{\boldsymbol{z}}[i][j] = \langle \boldsymbol{s}_1[i], cx^{j-1} \rangle$. We can obtain $\boldsymbol{s}_1$ by gathering faulty signatures. Islam et al. [IMS+22] also considered an invalid response attack, flipping a bit of $\boldsymbol{s}_1$ by using a rowhammer attack. They showed that each faulty signature reveals the value of the flipped bit.

Lowering Entropy Attack (LE): We additionally discuss a new variant of key-recovery attack, *lowering entropy attacks*, which reduces the entropy of $\boldsymbol{y}$ drastically. For example, Espitau et al. [EFGT16] used instruction skipping fault to make $\boldsymbol{y}$ sparse, e.g., $\boldsymbol{y} = (y, 0, \ldots, 0)$. In the case of BLISS, they obtain $\boldsymbol{z}_1 c^{-1} - \boldsymbol{s}_1 = c^{-1}\boldsymbol{y}_1 = \sum_{i=0}^{n'-1} y_{1,i} c^{-1} x^i \bmod q$. Thus, we obtain the partial information of $\boldsymbol{s}_1$ by solving CVP on a lattice spanned by $c^{-1}x^0, \ldots, c^{-1}x^{n'}$ and $q\mathbb{Z}^n$. Ulitzsch et al. [UMB+23] extended the attack by Espitau et al. to the implementation using a shuffling countermeasure. ElGhamrawy et al. [EAB+23] also proposed an instruction skipping fault, which skips the counter increments, to make multiple elements of $\boldsymbol{y}$ the same, e.g., $\boldsymbol{y} = (y, y, y_3, \ldots)$. This drastically reduces the dimension of the lattices to obtain partial information of $\boldsymbol{s}_1$.

B Fiat-Shamir with Aborts

We give the definitions of the Fiat-Shamir with aborts paradigm, denoted by FSwA, the deterministic variant [KLS18], denoted by DFSwA, and the hedged variant, denoted by HFSwA.[18] We show the algorithms of these paradigms in Fig. 13. The difference exists only in the way of taking per-loop randomness r. FSwA randomly chooses it. In contrast, DFSwA and HFSwA generate it using a PRF PRF, which is called hedged extractor in HFSwA. The signature generation of DFSwA is deterministic since the private key sk and a message m define the randomness. To make the signature generation probabilistic, HFSwA uses a nonce $n \in \mathcal{N}$ as an input for the hedged extractor.

C Missing Proofs

C.1 Proof of Lemma 4

We modify the proof of [AHU19, Theorem 3] to assume that H_0 and H_1 are adaptively reprogrammed and that the positions of differences between the two oracles, i.e., $\mathcal{S}_i$, change with each query.

Lemma 4 (O2H with Adaptive Reprogramming and Positioning – restated). *Let $\mathsf{H}_0, \mathsf{H}_1 \colon \mathcal{X} \to \mathcal{Y}$ be functions that are reprogrammed depending on the results of classical queries to an oracle O (H_0 and H_1 may be reprogrammed differently). Assume that $\mathsf{H}_0(x) = \mathsf{H}_1(x)$ for all $x \notin \mathcal{S}_i$, where $\mathcal{S}_i$ is defined*

[18] The original hedging [BT16, AOTZ20] generates the randomness by $\mathsf{H}'(sk \| m \| n)$, where H' is a random oracle, instead of $\mathsf{PRF}_K(m \| n)$.

based on the results of queries to O *up to the* i*-th query. Let* z *be a random bitstring. (*H_0*,* H_1*,* z *may have arbitrary joint distribution.) Let* $\mathcal{A}$ *be a quantum algorithm with* q *quantum queries to* H_0 *or* H_1 *and some classical queries to* O*. Let* $\mathcal{B}$ *be a quantum algorithm that, given access to the oracles* H_0 *and* O *and the adversary* $\mathcal{A}$*, performs the following steps on input* z*: picks* $i \leftarrow_{\$} [q]$*, runs* $\mathcal{A}$ *until the* i*-th query, measures the query input register in the computational basis, and outputs the measurement outcome* x *along with the index* i *along. Then, we have*

$$\left|\Pr\left[\mathcal{A}^{|\mathsf{H}_0\rangle,\mathrm{O}}(z)=1\right]-\Pr\left[\mathcal{A}^{|\mathsf{H}_1\rangle,\mathrm{O}}(z)=1\right]\right| \leq 2q\sqrt{\Pr\left[(i,x)\leftarrow\mathcal{B}^{|\mathsf{H}_0\rangle,\mathrm{O},\mathcal{A}}(z) : x\in\mathcal{S}_i\right]}.$$

Proof. This lemma is an adaptation of [AHU19, Theorem 3], which consists of [AHU19, Lemma 8] (proof assuming a pure state) and [AHU19, Lemma 9] (proof assuming a mixed state). We only modify the former and use the latter as it is.

First, we introduce the necessary notation. Let $\left|\psi^i_{\mathsf{H}_0}\right\rangle$ and $\left|\psi^i_{\mathsf{H}_1}\right\rangle$ be the states of $\mathcal{A}^{|\mathsf{H}_0\rangle,\mathrm{O}}(z)$ and $\mathcal{A}^{|\mathsf{H}_1\rangle,\mathrm{O}}(z)$ just before the $(i+1)$-th query, including the query input register. In [AHU19, Lemma 8], a bound on $D_i = \left\|\left|\psi^i_{\mathsf{H}_0}\right\rangle - \left|\psi^i_{\mathsf{H}_1}\right\rangle\right\|^2$ is obtained, from which $\sqrt{D_q} = \left\|\left|\psi^q_{\mathsf{H}_0}\right\rangle - \left|\psi^q_{\mathsf{H}_1}\right\rangle\right\|$ is derived. In this proof, in addition to the adversary's state, we define states $\left|\phi^i_{\mathsf{H}_0}\right\rangle$ and $\left|\phi^i_{\mathsf{H}_1}\right\rangle$ that store the results of queries to O by $\mathcal{A}^{|\mathsf{H}_0\rangle,\mathrm{O}}(z)$ and $\mathcal{A}^{|\mathsf{H}_1\rangle,\mathrm{O}}(z)$, respectively. Let O_{H_b} be a unitary operation corresponding to the oracle queries of H_b ($b \in \{0,1\}$) that returns $\mathsf{H}_b(x)$ by reading the query input register along with $\left|\phi^i_{\mathsf{H}_b}\right\rangle$. Note that H_b is reprogrammed depending on $\left|\phi^i_{\mathsf{H}_b}\right\rangle$, and $O_{\mathsf{H}_b}\left|\phi^i_{\mathsf{H}_b}\right\rangle|x,y\rangle = |x, y\oplus\mathsf{H}_b(x)\rangle$ holds. Define $P_{\mathcal{S}_i}$ as the orthogonal projector that projects the query register onto the subspace spanned by states $|x\rangle$ such that $x \in \mathcal{S}_i$, where $\mathcal{S}_i$ is also defined by O_{H_b}. Formally, this is given by $P_{\mathcal{S}_i} = \sum_{x\in\mathcal{S}_i}|x\rangle\langle x|$. $P_{\mathcal{S}_i}$ ensures that only those states corresponding to $x \in \mathcal{S}_i$ are selected. Let $B_i = \left\|P_{\mathcal{S}_i}\left|\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\|^2$ be the probability of obtaining $x \in \mathcal{S}_i$ by measuring the query input register of $\mathcal{A}^{|\mathsf{H}_0\rangle,\mathrm{O}}(z)$ just before the i-th query. Let $D'_i = \left\|\left|\phi^i_{\mathsf{H}_0}\psi^i_{\mathsf{H}_0}\right\rangle - \left|\phi^i_{\mathsf{H}_1}\psi^i_{\mathsf{H}_1}\right\rangle\right\|^2$, and let U be the state transition operation between queries to H_b, where U includes queries to O.

Then, we obtain a bound on $\left\|\left|\psi^q_{\mathsf{H}_0}\right\rangle - \left|\psi^q_{\mathsf{H}_1}\right\rangle\right\|$ by deriving D'_i as follows.

$$\begin{aligned}
D'_i &= \left\|UO_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - UO_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right\|^2 \\
&\overset{(*)}{=} \left\|O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right\|^2 \\
&= \left\|\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right) + \left(O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right)\right\|^2 \\
&\overset{(**)}{\leq} \left\|\left(O_{\mathsf{H}_0} - O_{\mathsf{H}_1}\right)\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\|^2 + \left\|O_{\mathsf{H}_1}\left(\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right)\right\|^2 \\
&\quad + 2\left\|\left(O_{\mathsf{H}_0} - O_{\mathsf{H}_1}\right)\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\| \cdot \left\|O_{\mathsf{H}_1}\left(\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right)\right\| \\
&= \left\|\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)\left|\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\|^2 + \left\|O_{\mathsf{H}_1}\left(\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right)\right\|^2 \\
&\quad + 2\left\|\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)\left|\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\| \cdot \left\|O_{\mathsf{H}_1}\left(\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right)\right\| \\
&\overset{(***)}{\leq} \left\|\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)P_{\mathcal{S}_i}\left|\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\|^2 + \left\|O_{\mathsf{H}_1}\left(\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right)\right\|^2 \\
&\quad + 2\left\|\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)P_{\mathcal{S}_i}\left|\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\| \cdot \left\|O_{\mathsf{H}_1}\left(\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i-1}_{\mathsf{H}_1}\psi^{i-1}_{\mathsf{H}_1}\right\rangle\right)\right\| \\
&\overset{(****)}{\leq} 4\left\|P_{\mathcal{S}_i}\left|\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\|^2 + \left\|\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i}_{\mathsf{H}_1}\psi^{i}_{\mathsf{H}_1}\right\rangle\right\|^2 + 4\left\|P_{\mathcal{S}_i}\left|\psi^{i-1}_{\mathsf{H}_0}\right\rangle\right\| \cdot \left\|\left|\phi^{i-1}_{\mathsf{H}_0}\psi^{i-1}_{\mathsf{H}_0}\right\rangle - \left|\phi^{i}_{\mathsf{H}_1}\psi^{i}_{\mathsf{H}_1}\right\rangle\right\| \\
&= 4B_i + D'_{i-1} + 4\sqrt{B_iD'_{i-1}} = \left(\sqrt{D'_{i-1}} + 2\sqrt{B_i}\right)^2
\end{aligned} \tag{5}$$

As in [AHU19, Lemma 8], we use the unitarity of U in $(*)$, the triangle inequality in $(**)$, and the fact that $\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)$ has an operator norm of at most 2 in $(****)$. We show that the inequality of $(***)$ holds. Since O_{H_0} and O_{H_1} both read the same state $\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle$, it follows that $\mathsf{H}_0(x) = \mathsf{H}_1(x)$ for all $x \notin \mathcal{S}_i$. Therefore, $\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)$ only applies to $|x,y\rangle$ such that $x \in \mathcal{S}_i$. Consequently, even if $P_{\mathcal{S}_i}$ projects the query input register onto the subspace spanned by $|x\rangle$ for $x \in \mathcal{S}_i$, the state remains unchanged after applying $\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)$. Therefore, we have

$$\left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right)P_{\mathcal{S}_i} = \left(O_{\mathsf{H}_0}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle - O_{\mathsf{H}_1}\left|\phi^{i-1}_{\mathsf{H}_0}\right\rangle\right).$$

From Eq. (5) and $D'_0 = 0$, we have

$$\sqrt{D'_q} \le 2\sum_{i\in[q]}\sqrt{B_i} \le 2q\sqrt{\sum_{i\in[q]}\frac{1}{q}B_i} = 2q\sqrt{\Pr\left[(i,x) \leftarrow \mathcal{B}^{|\mathsf{H}_0\rangle,\mathrm{O},\mathcal{A}}(z) : x \in \mathcal{S}_i\right]}.$$

Since $\sqrt{D_q} \le \sqrt{D'_q}$ holds, we have

$$\left\||\psi^q_{\mathsf{H}_0}\rangle - |\psi^q_{\mathsf{H}_1}\rangle\right\| = \sqrt{D_q} \le 2q\sqrt{\Pr\left[(i,x) \leftarrow \mathcal{B}^{|\mathsf{H}_0\rangle,\mathrm{O},\mathcal{A}}(z) : x \in \mathcal{S}_i\right]}.$$

Extending this result to O2H for mixed states using the proof of [AHU19, Lemma 9], we have this lemma. □

C.2 Proof of Lemma 5

Lemma 5 (Collison Resistance with Fault Injection against Random Function – restated). *Let* $\mathsf{H}\colon \mathcal{X} \to \mathcal{Y}$ *be a random function. Then, any quantum algorithm that makes* q *quantum queries to* H *outputs a pair such that* $\mathsf{H}(x) = \phi(\mathsf{H}(x'))$ *with probability at most* $632 \cdot 2^t \binom{\log_2(|\mathcal{Y}|)+1}{t}(q+1)^3/|\mathcal{Y}|$*, and a pair such that* $\phi(\mathsf{H}(x)) = \phi'(\mathsf{H}(x'))$ *with probability at most* $632 \cdot 2^{2t}\binom{\log_2(|\mathcal{Y}|)+1}{2t}(q+1)^3/|\mathcal{Y}|$*, where* $\phi, \phi' \in \Phi$*.*

Proof. We construct a reduction from standard collision resistance to prove this lemma. Let $\mathcal{A}_{\mathsf{crf}}$ be an adversary with quantum access to H that attempts to find a collision with fault injection, and let $\mathcal{B}_{\mathsf{cr}}$ be an adversary for finding a collision in a random function $\mathsf{H}' \leftarrow_{\$} \mathsf{Func}(\mathcal{X}, \mathcal{Y}')$.

First, we consider the case where $\mathcal{A}_{\mathsf{crf}}$ attempts to find (ϕ, x, x') such that $\mathsf{H}(x) = \phi(\mathsf{H}(x'))$, with a winning probability of ϵ. Assuming $|\mathcal{Y}'| = |\mathcal{Y}| \cdot 2^{-t}$, we show that $\mathcal{B}_{\mathsf{cr}}$ can win its game with a probability of at least $\epsilon \cdot \binom{\log_2(|\mathcal{Y}|)+1}{t}^{-1}$, which establishes the reduction. The strategy of $\mathcal{B}_{\mathsf{cr}}$ is as follows. Given quantum access to H', $\mathcal{B}_{\mathsf{cr}}$ simulates $\mathcal{A}_{\mathsf{crf}}$ by first selecting t bit positions of $\mathsf{H}'(x)$ and $\mathsf{G} \leftarrow_{\$} \mathsf{Func}(\mathcal{X}, \{0,1\}^t)$. To compute $\mathsf{H}(x)$, $\mathcal{B}_{\mathsf{cr}}$ queries $\mathsf{H}'(x)$, computes $\mathsf{G}(x)$, and inserts t bits of $\mathsf{G}(x)$ into the selected t positions of $\mathsf{H}'(x)$. Note that this computation can be performed in superposition. When $\mathcal{A}_{\mathsf{crf}}$ submits (ϕ, x, x'), $\mathcal{B}_{\mathsf{cr}}$ submits (x, x'). Conditioned on $\mathcal{A}_{\mathsf{crf}}$ winning, $\mathcal{B}_{\mathsf{cr}}$ wins its game with probability at least $\binom{\log_2(|\mathcal{Y}|)+1}{t}^{-1}$, since the randomly selected positions match those where ϕ is applied with at least this probability. Thus, $\mathcal{B}_{\mathsf{cr}}$ can win the game with probability at least $\epsilon \cdot \binom{\log_2(|\mathcal{Y}|)+1}{t}^{-1}$. From [Zha15, Theorem 3.1], $\mathcal{B}_{\mathsf{cr}}$'s winning probability is at most $632(q+1)^3/|\mathcal{Y}'|$ [19]. Since $|\mathcal{Y}'| = |\mathcal{Y}| \cdot 2^{-t}$, $\epsilon \cdot \binom{\log_2(|\mathcal{Y}|)+1}{t}^{-1} \le 632 \cdot 2^t(q+1)^3/|\mathcal{Y}|$ holds. Thus, we have the bound $632 \cdot 2^t \cdot \binom{\log_2(|\mathcal{Y}|)+1}{t}(q+1)^3/|\mathcal{Y}|$.

Next, we consider the case where $\mathcal{A}_{\mathsf{crf}}$ finds a pair satisfying $\phi(\mathsf{H}(x)) = \phi'(\mathsf{H}(x'))$, with a winning probability of ϵ'. Assuming $|\mathcal{Y}'| = |\mathcal{Y}| \cdot 2^{-2t}$, the advantage of $\mathcal{B}_{\mathsf{cr}}$ can similarly bound the advantage of $\mathcal{A}_{\mathsf{crf}}$. $\mathcal{B}_{\mathsf{cr}}$ randomly selects the positions of $2t$ bits to which ϕ and ϕ' are applied. As in the case where there is no fault on one side, we have $\epsilon' \cdot \binom{\log_2(|\mathcal{Y}|)+1}{2t}^{-1} \le 632 \cdot 2^{2t}(q+1)^3/|\mathcal{Y}|$, and thus the bound is $632 \cdot 2^{2t} \cdot \binom{\log_2(|\mathcal{Y}|)+1}{2t}(q+1)^3/|\mathcal{Y}|$. □

C.3 Proof of Lemma 6

Lemma 6 (Special No-Abort HVZK of Simplified ML-DSA – restated). *Let* ID *be the underlying ID scheme of the simplified ML-DSA. Then,* ID *is snaHVZK.*

Proof. We show that the output distribution of Trans is identical to the one of Sim for any valid key pair (pk', sk') and $c \in B_\tau$ by following [KLS18, Lemma 4.3] (see the definitions of Trans and Sim in Fig. 9). The proof slightly differs in how the challenge is handled, the distribution of $\boldsymbol{y}$, and an additional condition for rejection (see Line 14 of Trans); in their case, the challenge is chosen by Trans and Sim and the range of $\boldsymbol{y}$ is symmetric $S^\ell_{\gamma_1}$.

[19] Kulkarni and Xagawa derived the universal constant as 632 in [KX24b].

Fix $\boldsymbol{z} \in S^{\ell}_{\gamma_1-\beta-1}$. We compute the probability that $\boldsymbol{z}$ is generated by line 7 of Trans. For any $c \in B_\tau$, we have

$$\Pr_{\boldsymbol{y} \leftarrow\!\$ (S'_{\gamma_1})^{\ell}}[\boldsymbol{z} = \boldsymbol{y} + c\boldsymbol{s}_1] = \Pr_{\boldsymbol{y} \leftarrow\!\$ (S'_{\gamma_1})^{\ell}}[\boldsymbol{y} = \boldsymbol{z} - c\boldsymbol{s}_1].$$

Since $c \in B_\tau$, $\|\boldsymbol{s}_1\|_\infty \le \eta$, and $\beta = \tau\eta$, we have always $\|c\boldsymbol{s}_1\|_\infty \le \beta$ and $\boldsymbol{z} - c\boldsymbol{s}_1 \in S^{\ell}_{\gamma_1-1} \subseteq (S'_{\gamma_1})^{\ell}$. Hence, we also have

$$\Pr_{\boldsymbol{y} \leftarrow\!\$ (S'_{\gamma_1})^{\ell}}[\boldsymbol{y} = \boldsymbol{z} - c\boldsymbol{s}_1] = \frac{1}{|(S'_{\gamma_1})^{\ell}|}.$$

Thus, combining those equations, every $\boldsymbol{z} \in S^{\ell}_{\gamma_1-\beta-1}$ has the same probability of being generated just after the if statement Line 8 of Trans, that is, $1/|(S'_{\gamma_1})^{\ell}|$. In addition, this shows that

$$\Pr[\|\boldsymbol{z}\|_\infty \ge \gamma_1 - \beta] = 1 - \frac{|S^{\ell}_{\gamma_1-\beta-1}|}{|(S'_{\gamma_1})^{\ell}|} = 1 - \left(\frac{2\gamma_1 - 2\beta + 1}{2\gamma_1}\right)^{256\ell}, \tag{6}$$

at Line 8.

In contrast, Sim outputs $\perp$ with the same probability as Eq. (6) in Line 18; otherwise, it chooses $\boldsymbol{z}$ from $S^{\ell}_{\gamma_1-\beta-1}$ uniformly at random in Line 22. Notice that we have $\boldsymbol{w} - c\boldsymbol{s}_2 = \boldsymbol{A}\boldsymbol{z} - c\boldsymbol{t}$ as in [KLS18]. By this identification, subsequent computations in Trans are derived from $\boldsymbol{z}$, c, and pk' and are identical to those in Sim, and the output distribution of Trans is identical to that of Sim. □

D Intuitive Explanations on Fault-resilience of ML-DSA

We provide an intuitive explanation of the faults that can be prevented in ML-DSA.

- $f_{\mathsf{H}_\mu,\mathsf{i}}$, $f_{\mathsf{H}_\mu,\mathsf{o}}$, $f_{\mathsf{H}_{\mathsf{he}},\mathsf{i}1}$, $f_{\mathsf{H}_{\mathsf{he}},\mathsf{o}}$, and $f_{\mathsf{H}_{\mathsf{em}},\mathsf{i}2,i}$: Since the randomness of $\boldsymbol{y}$ is preserved, faults introduced via these functions do not compromise the security of ML-DSA. Faults injected via $f_{\mathsf{H}_\mu,\mathsf{i}}$ and $f_{\mathsf{H}_\mu,\mathsf{o}}$ do not compromise the uniqueness of the pair (rnd, μ), assuming $(\overline{M'}, rnd) \notin \mathcal{Q}_{M',rnd}$. Provided that the input to H_{he} is unique, the output ρ'' retains its randomness even if a fault is introduced into part of the secret key through $f_{\mathsf{H}_{\mathsf{he}},\mathsf{i}1}$. Moreover, as long as the counter used in $\mathsf{H}^{(\ell)}_{\mathsf{em}}$ ensures uniqueness, the output $\boldsymbol{y}$ remains random, even when ρ'' is affected by faults via $f_{\mathsf{H}_{\mathsf{he}},\mathsf{o}}$ or $f_{\mathsf{H}_{\mathsf{em}},\mathsf{i}2,i}$. In this way, the randomness of $\boldsymbol{y}$ is preserved despite the above fault injections.
- $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}$, $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}$, $f_{\mathsf{Sb},\mathsf{i},i}$, $f_{\mathsf{Ab},\mathsf{o},i}$: The commitment and response are correctly generated even in the presence of faults introduced via these functions. P_1 takes fault-free $\boldsymbol{y}$ and ρ as input, ensuring that the output $(\boldsymbol{w}_1, st)$ is correctly generated. For the challenge hash H_{ch}, as long as $\boldsymbol{w}_1$ possesses sufficient min-entropy, the resulting $\tilde{c}$ remains random, even under a fault via $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{i},i}$. Even though $\tilde{c}$ is tampered through $f_{\mathsf{H}_{\mathsf{ch}},\mathsf{o},i}$ or $f_{\mathsf{Sb},\mathsf{i},i}$, the sampling function Sb can still return a valid challenge $c \in B_\tau$. Since P_2 receives a valid challenge c and fault-free secret and state, the response $(\boldsymbol{z}, \boldsymbol{h})$ is correctly generated. Furthermore, even if $(\boldsymbol{z}, \boldsymbol{h})$ is subsequently tampered with via $f_{\mathsf{Ab},\mathsf{o},i}$, the result remains simulatable under the snaHVZK assumption, and such faults therefore cannot be exploited for a successful attack.

E Concrete Analysis of Parameter Sets

Summarizing Theorems 1 and 2, the following security bound is obtained.

$$\begin{aligned}\mathrm{Adv}^{\text{EUF-fCMNA}}_{\text{ML-DSA}}(\mathcal{A}_{\mathsf{cmna}}) \le \mathrm{Adv}^{\text{EUF-NMA}}_{\text{sML-DSA}}(\mathcal{A}_{\mathsf{nma}}) &+ \frac{3}{2}q'_{\mathsf{S}}\sqrt{\frac{q_{\mathsf{H}_{\mathsf{ch}}} + q'_{\mathsf{S}} + 1}{2^{\alpha-t}}} + 2q_{\mathsf{H}_{\mathsf{ch}}}\sqrt{\frac{q'_{\mathsf{S}}}{2^{\alpha-t}}} + \frac{\binom{513}{t}(q_{\mathsf{H}_\mu}+1)^3}{2^{502-t}} \\ &+ \frac{\binom{513}{2t}(q_{\mathsf{H}_\mu}+1)^3}{2^{501-2t}} + \frac{\sqrt{\sum_{i\in[t+1]}\binom{256}{i-1}}(q_{\mathsf{H}_{\mathsf{he}}}+1)}{2^{127}} + \frac{\sqrt{\sum_{i\in[t+1]}\binom{512}{i-1}q_{\mathsf{S}}}(q_{\mathsf{H}_{\mathsf{em}}}+1)}{2^{255}} \\ &+ \frac{q_{\mathsf{H}_{\mathsf{es}}}}{2^{127}} + \frac{\sum_{i\in[2t+1]}\binom{\mathsf{bitlen}(\boldsymbol{w}_1)}{i-1}{q_{\mathsf{S}}}^2}{2^{\alpha}} + \frac{\sum_{i\in[2t+1]}\binom{512}{i-1}{q_{\mathsf{S}}}^2}{2^{512}} + 4\delta.\end{aligned}$$

For the parameters (α, δ), where the commitment has min-entropy α with probability $1-\delta$ (see the exact definition in Definition 2), Barbosa et al. [BBD+23] established the correspondence between α and δ in

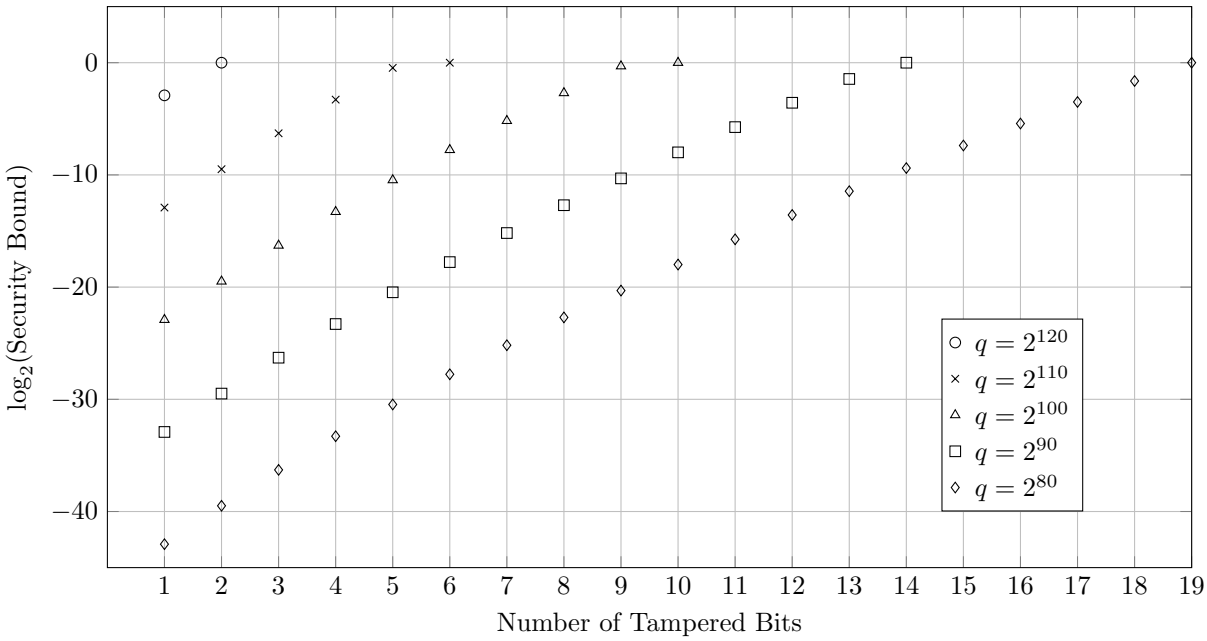

Fig. 14: Security bound excluding the EUF-NMA advantage under varying the number of tampered bits

[BBD+23, Figure 11]. We can adjust (α, δ) using the table. For instance, when we set $\delta = 2^{-135}$, we have $(\alpha_{44}, \alpha_{65}, \alpha_{87}) = (310, 1120, 1633)$, where α_{44}, α_{65}, and α_{87} represents the min-entropy of ML-DSA-44, ML-DSA-65, and ML-DSA-87, respectively.

Assuming $q = q_{\mathsf{H}_{\mathsf{ch}}} = q_{\mathsf{H}_{\mathsf{he}}} = q_{\mathsf{H}_{\mathsf{em}}} = q_{\mathsf{S}}$, we plot the security bound excluding $\mathrm{Adv}^{\text{EUF-NMA}}_{\mathsf{sML\text{-}DSA}}(\mathcal{A}_{\mathsf{nma}})$ in Fig. 14. Notably, the dominant terms in the bound are $\sqrt{\sum_{i\in[t+1]}\binom{256}{i-1}}(q_{\mathsf{H}_{\mathsf{he}}}+1)\cdot 2^{-127}$ and $q_{\mathsf{H}_{\mathsf{es}}}\cdot 2^{-127}$ across all parameter sets, which significantly outweighs the contribution of the remaining terms.